

# ***MCU & FPGA Based Driver Drowsiness Detection using BNN***

## **Nhóm : 06- EDABK -TVD**

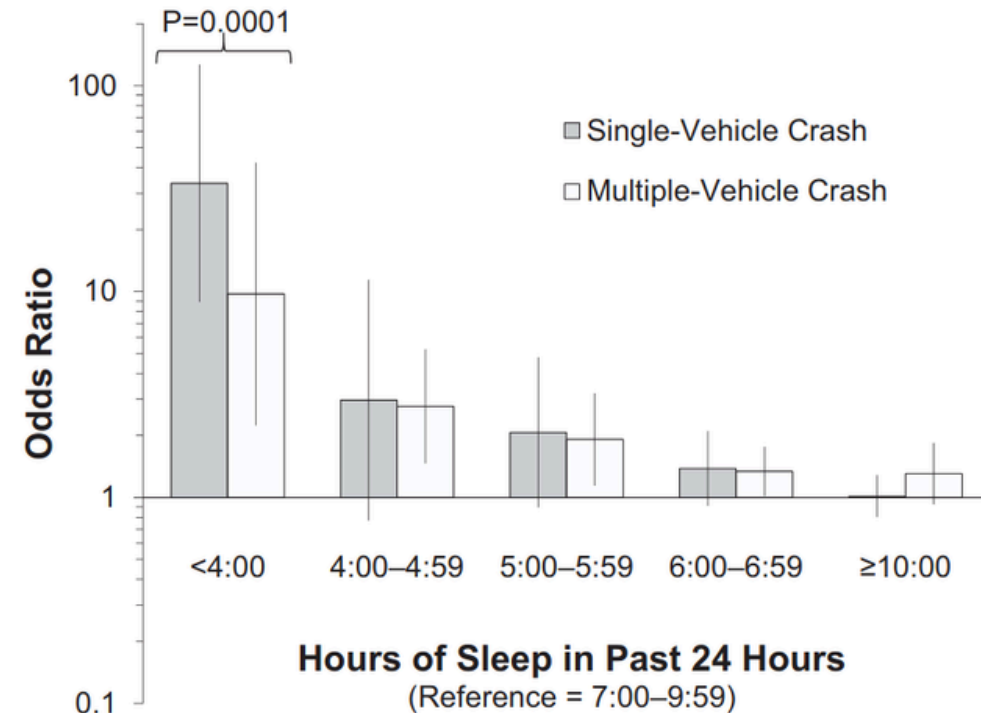
- Hoàng Văn Thái
- Tạ Văn Việt
- Nguyễn Minh Đức
- Dương Phan Quang Bách

**GVHD:** PGS. Nguyễn Đức Minh

- 1. Giới thiệu đề tài.**
- 2. Chỉ tiêu kĩ thuật.**
- 3. Kiến trúc hệ thống.**
- 4. Luồng dữ liệu mạng BNN.**
- 5. Kiểm thử hệ thống.**
- 6. Kết quả và hướng phát triển.**

# 1. GIỚI THIỆU ĐỀ TÀI

## • Đặt vấn đề:



**Table 2.** Odds ratios for driver culpability in relation to total hours of sleep in 24 hr before crash and fatigue-related covariates

|                                               | Odds ratio (95% confidence interval) |
|-----------------------------------------------|--------------------------------------|
| Hours of sleep in 24 hr before crash          |                                      |
| <4:00                                         | 15.1 (4.2–54.4)                      |
| 4:00–4:59                                     | 2.9 (1.4–6.2)                        |
| 5:00–5:59                                     | 1.9 (1.1–3.2)                        |
| 6:00–6:59                                     | 1.3 (1.04–1.7)                       |
| 7:00–9:59                                     | 1 (Reference)                        |
| ≥10:00                                        | 1.3 (0.9–1.7)                        |
| Changed sleep or work schedule in past 7 days |                                      |
| Yes                                           | 1.3 (1.1–1.6)                        |
| No                                            | 1 (Reference)                        |
| Typically feels fatigued/drowsy upon waking   |                                      |
| Yes                                           | 2.2 (1.5–3.2)                        |
| No                                            | 1 (Reference)                        |

Mối quan hệ giữa thời gian ngủ trước khi lái xe và tỉ lệ gây tai nạn.

Tefft, B. C. (2018). Acute sleep deprivation and culpable motor vehicle crash involvement. *Sleep*, 41(10). <https://doi.org/10.1093/sleep/zsy144>

# 1. GIỚI THIỆU ĐỀ TÀI

- **Giải pháp đề xuất: Hệ thống cảnh báo ngủ gật khi lái xe**
  - **Công nghệ:** Nhận dạng khuôn mặt/ mắt bằng AI để đánh giá độ tập trung lúc lái xe
  - **Phần cứng:** Xử lý trên MCU Sonix SNF407 và FPGA OneKiwi 1P5
  - **Cơ chế cảnh báo:** Phản hồi bằng âm thanh





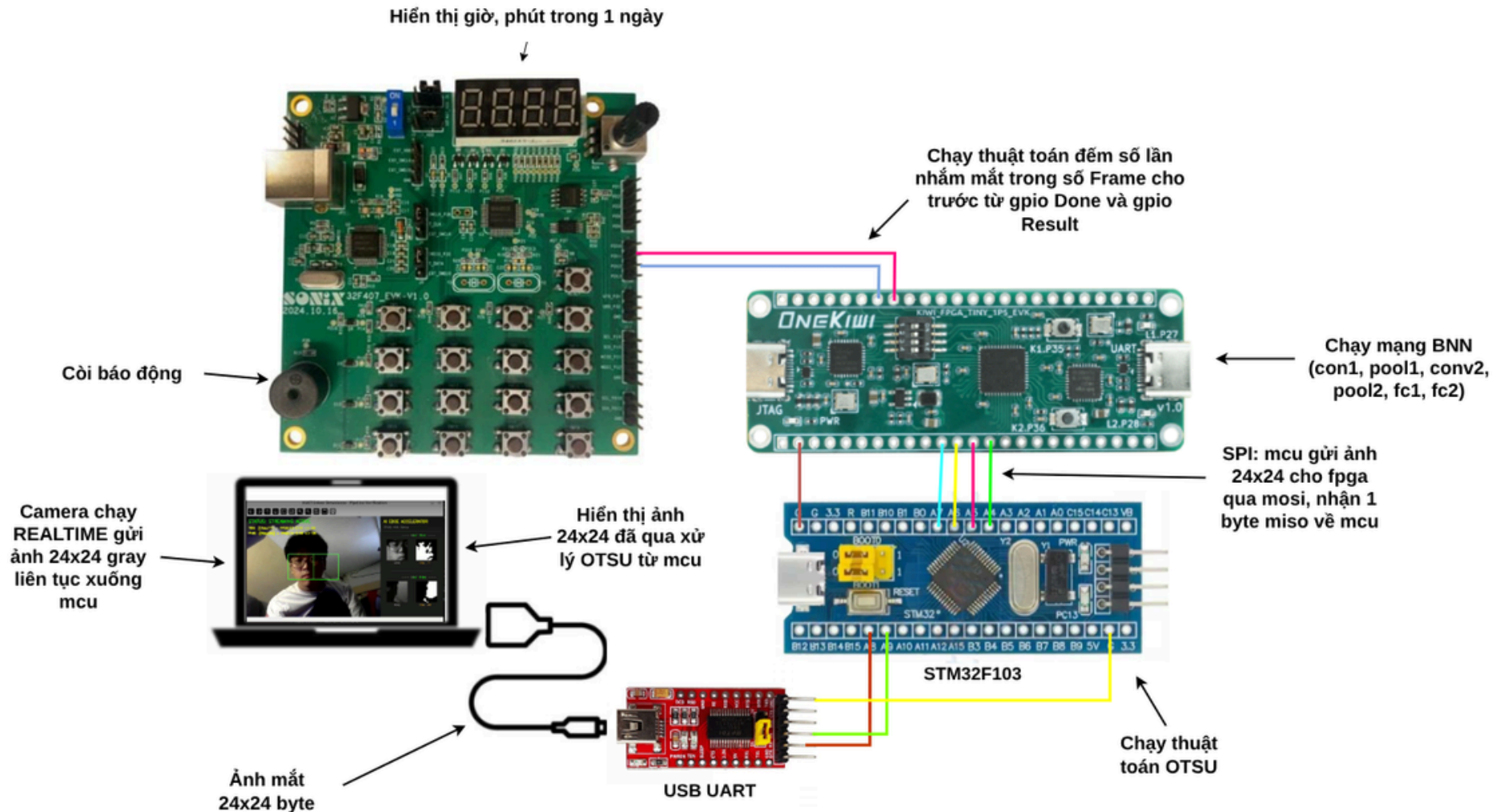
## 2. Chỉ tiêu kỹ thuật

- SPEC**

| Hạng mục phần cứng / Giao tiếp              | Thông số Kỹ thuật (SPEC)                                                                           |
|---------------------------------------------|----------------------------------------------------------------------------------------------------|
| Tần số lõi FPGA                             | 27 MHz                                                                                             |
| Tài nguyên Logic (LUT & FF)                 | Tổng hợp $\leq 1584$ LUT                                                                           |
| Weight, ngưỡng & Bram lưu tensor từng layer | BSRAM $\leq 72K$ bits (4BSRAM)                                                                     |
| Giao tiếp MCU $\leftrightarrow$ FPGA        | Truyền khung ảnh 72 bytes. MCU làm Master (MOSI), FPGA làm Slave (MISO trả về Popcount & cờ Busy). |
| Cơ chế Đồng bộ (Handshake)                  | Đảm bảo PC chỉ gửi ảnh khi MCU đã sẵn sàng nhận dữ liệu qua UART.                                  |
| Giao tiếp Cảnh báo                          | P3.13 (Ngắt), P3.12 (Logic)                                                                        |
| Thuật toán Báo động                         | Còi chip (Buzzer)                                                                                  |

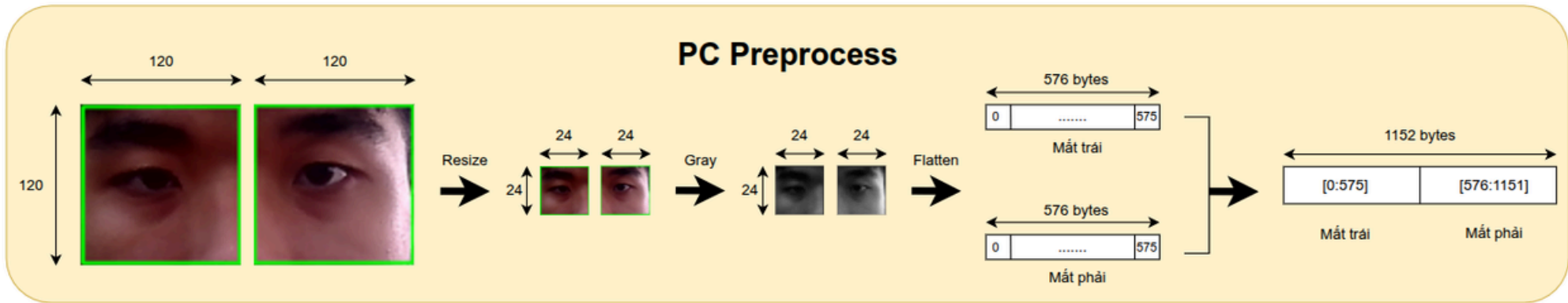
## 2. Chỉ tiêu kĩ thuật

### • Quá trình hoạt động Realtime:



### 3. Kiến trúc hệ thống

- Kiến trúc phần mềm**

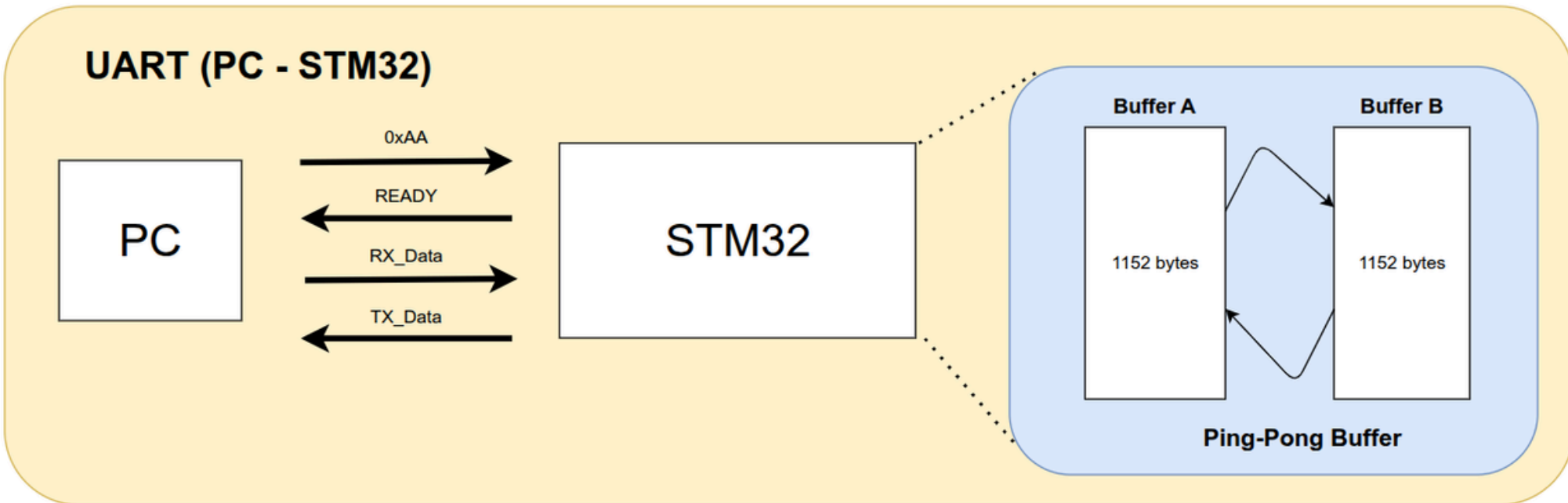


- Chức năng của camera (Giả lập trên PC)**

- + **Thu thập & Trích xuất (Crop):** Thu nhận luồng video thực tế và trích xuất vùng quan tâm (ROI) chứa mắt trái và mắt phải với kích thước 120 x 120 pixel.
- + **Tiền xử lý (Preprocess):** Chuyển đổi ảnh sang dạng xám (Grayscale) và nội suy (Resize) thu nhỏ về kích thước 24 x 24 pixel để tối ưu khối lượng tính toán cho phần cứng.
- + **Trải phẳng dữ liệu (Flatten):** Chuyển đổi ma trận ảnh 2D thành mảng 1 chiều (1D), mỗi mắt chiếm 576 bytes dữ liệu.
- + **Đóng gói & Truyền dẫn:** Nối chuỗi mảng mắt trái và mắt phải thành một gói tin (Payload) hoàn chỉnh có kích thước 1152 bytes và stream liên tục xuống MCU qua UART.

### 3. Kiến trúc hệ thống

- **Giao tiếp UART (PC - STM32) & Cơ chế Ping-Pong Buffer**

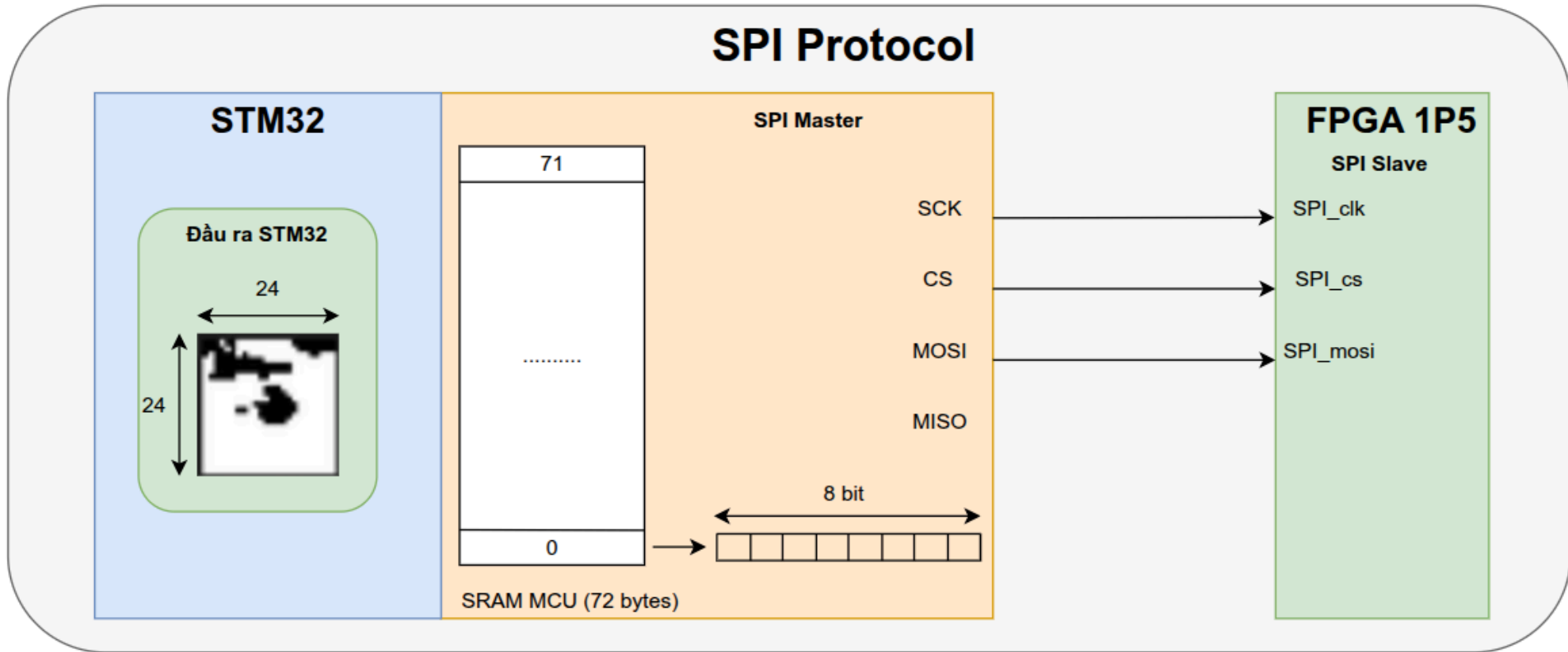


- **Chức năng:**

- + **Tốc độ truyền:** Baudrate 115200 bps (đảm bảo stream ảnh liên tục).
- + **Đồng bộ (Handshake):** PC gửi 0xAA → STM32 phản hồi READY để bắt đầu nhận data.
- + **Ping-Pong Buffer (Non-blocking):** Sử dụng 2 bộ đệm luân phiên (Buffer\_A và Buffer\_B, 1152 bytes/mảng). Cho phép STM32 vừa xử lý khung hình cũ (tính OTSU, đẩy SPI sang FPGA) vừa hứng khung hình mới cùng lúc.
- + **Phản hồi (Feedback):** STM32 trả ngược về PC các dữ liệu (Ngưỡng Otsu, Popcount, Result, chuỗi Hex) để đối chiếu kết quả.

### 3. Kiến trúc hệ thống

- **Giao tiếp SPI (STM32 - FPGA)**



- **Chức năng:**

+ **Cấu trúc dữ liệu (Payload):** Ảnh nhị phân kích thước 24 x 24 pixel (tương đương 576 bits) nén chặt thành 72 Bytes.

+ **Tiến trình truyền SPI (MOSI):** Kéo CS xuống Low (0) -> Truyền 72 bytes dữ liệu (MSB First) -> kéo CS lên High (1) sau byte thứ 72 để chốt khung hình (Frame)

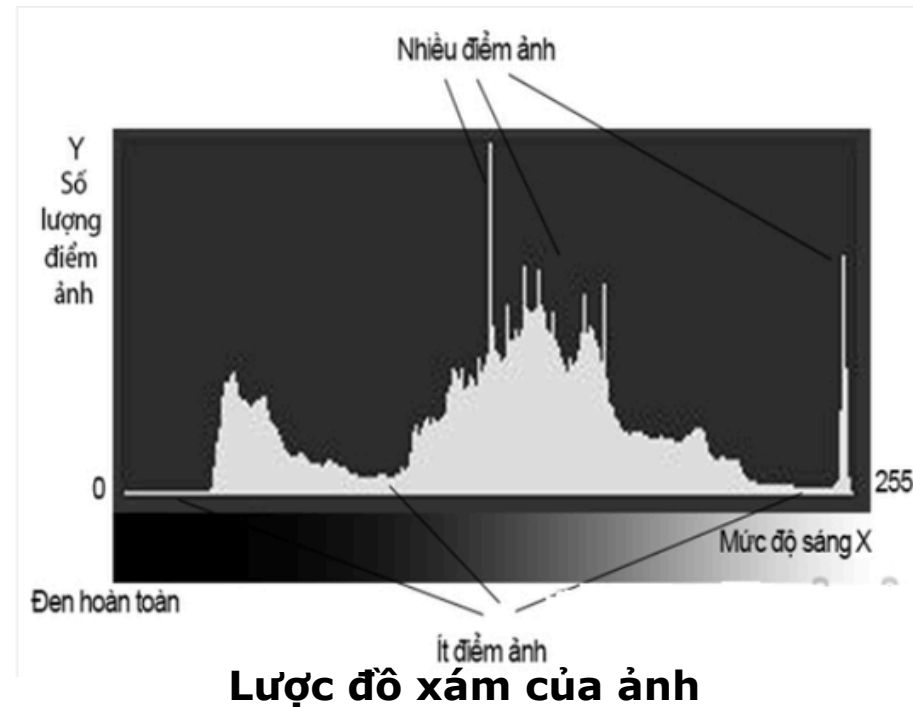
### 3. Kiến trúc hệ thống

#### • Thuật toán Otsu - Tự động tìm ngưỡng nhị phân (Auto-Thresholding)

+ **Vấn đề:** Điều kiện ánh sáng khi lái xe thay đổi liên tục, nếu dùng một mức ngưỡng (Threshold) cố định để nhị phân hóa ảnh mắt sẽ dẫn đến sai số lớn.

+ **Giải pháp:** Áp dụng thuật toán Otsu để phân tích biểu đồ tần suất (Histogram) của ảnh xám 24 x 24 và tự động tìm ra ngưỡng cắt tối ưu nhất.

+ **Mục tiêu của Otsu:** Tìm ra một giá trị ngưỡng  $T$  (từ 0 đến 255) sao cho ảnh được chia làm 2 lớp: Lớp nền (Background - Pixel nhóm đen) và Lớp đối tượng (Foreground - Pixel nhóm trắng) có sự khác biệt (phương sai) lớn nhất.



### 3. Kiến trúc hệ thống

- **Cơ sở Toán học: Phương sai giữa hai lớp (Between-Class Variance)**

+ Thuật toán duyệt qua mọi giá trị ngưỡng  $t$  để tìm giá trị làm cực đại hàm phương sai :

$$\sigma^2 = W_B \cdot W_F \cdot (\mu_B - \mu_F)^2$$

+ Chú thích:

- Trọng số (Tỉ lệ pixel) của nhóm Đen (Background) và nhóm Trắng (Foreground):

$$W_B = \frac{\text{weight B}}{\text{total pixel}}$$

$$W_F = \frac{\text{weight F}}{\text{total pixel}}$$

- Giá trị mức xám trung bình của nhóm Đen và nhóm Trắng:

$$\mu_B = \frac{\text{sum B}}{\text{weight B}}$$

$$\mu_F = \frac{\text{sum F}}{\text{weight F}}$$

### 3. Kiến trúc hệ thống

- **Tối ưu hóa thuật toán Otsu trên STM32**

+ Để tăng tốc độ xử lý cho hệ thống Real-time, phương trình gốc được biến đổi toán học nhằm loại bỏ các phép tính phân số phức tạp:

- **Khai triển phương trình gốc:**

$$\sigma^2 = \frac{\text{weight B} \cdot \text{weight F}}{(\text{total pixel})^2} \cdot \left( \frac{\text{sum B}}{\text{weight B}} - \frac{\text{sum F}}{\text{weight F}} \right)^2$$

- **Quy đồng và rút gọn:**

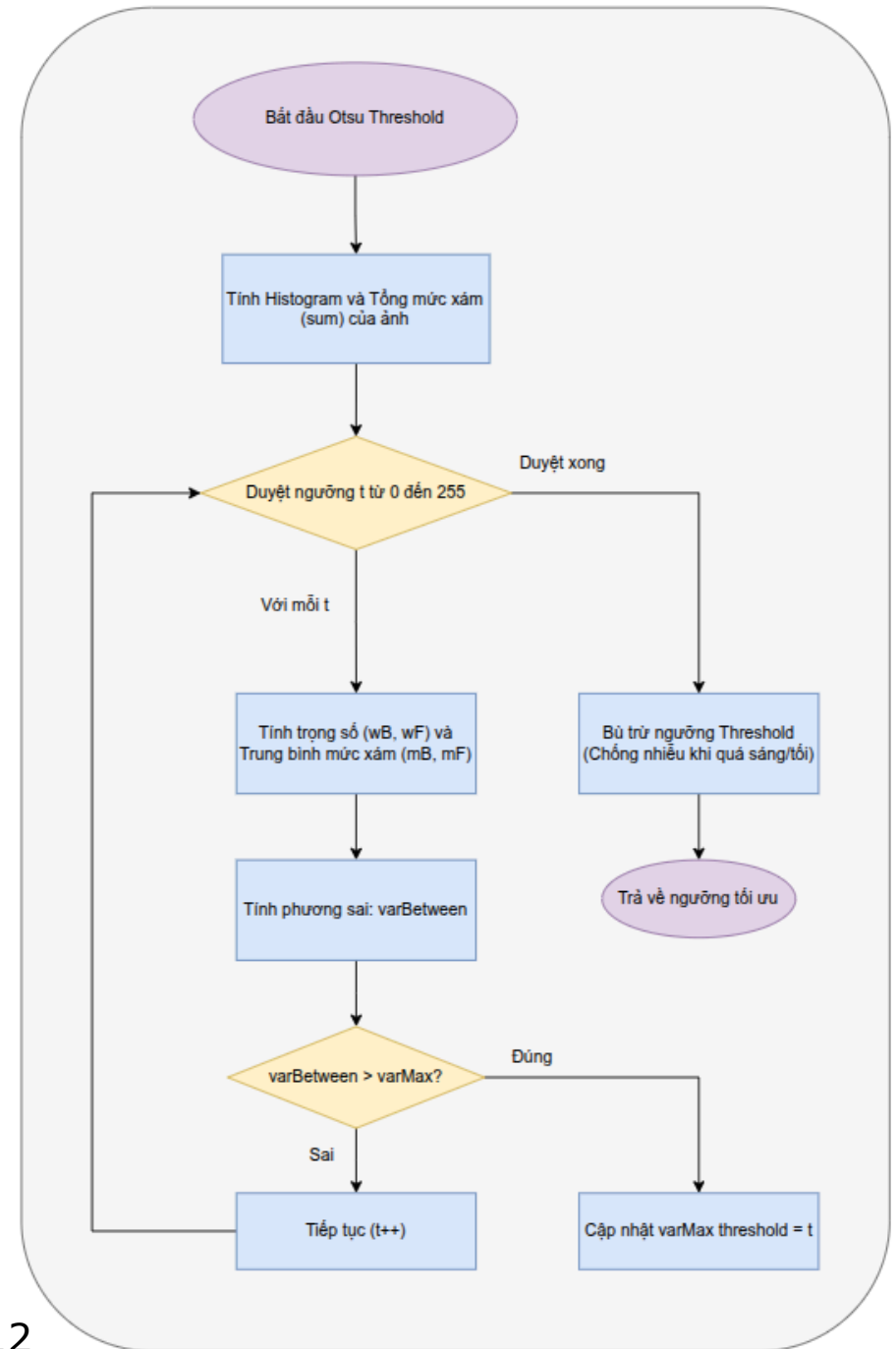
$$\sigma^2 = \frac{1}{(\text{total pixel})^2} \cdot \frac{(\text{sum B} \cdot \text{weight F} - \text{sum F} \cdot \text{weight B})^2}{\text{weight B} \cdot \text{weight F}}$$

$$\begin{aligned} \Rightarrow \sigma^2 &= \frac{\text{weight B} \cdot \text{weight F}}{(\text{total pixel})^2} \cdot \left( \frac{\text{sum B}}{\text{weight B}} - \frac{\text{sum F}}{\text{weight F}} \right)^2 \\ &= \frac{\text{weight B} \cdot \text{weight F}}{(\text{total pixel})^2} \cdot \left( \frac{\text{sum B} \cdot \text{weight F} - \text{sum F} \cdot \text{weight B}}{\text{weight B} \cdot \text{weight F}} \right)^2 \\ &= \frac{1}{(\text{total pixel})^2} \cdot \frac{(\text{sum B} \cdot \text{weight F} - \text{sum F} \cdot \text{weight B})^2}{\text{weight B} \cdot \text{weight F}} \\ &= \frac{1}{(\text{total pixel})^2} \cdot \frac{(\text{diff})^2}{\text{den}} \leftarrow \begin{array}{l} \text{Hằng số} = \text{Const} \\ \text{Tính} \end{array} \end{aligned}$$



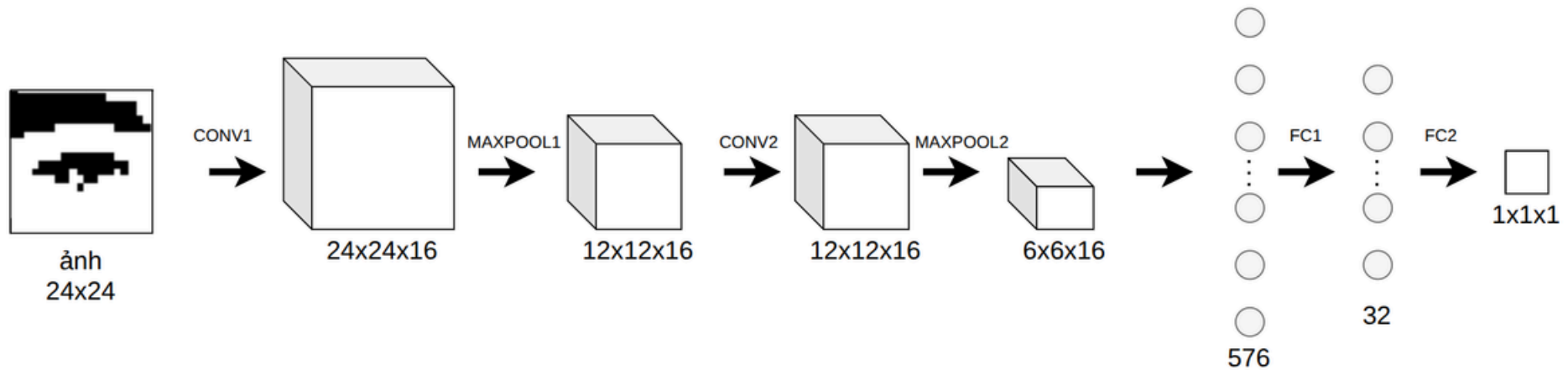
### 3. Kiến trúc hệ thống

- Lưu đồ thực thi Otsu trên phần cứng



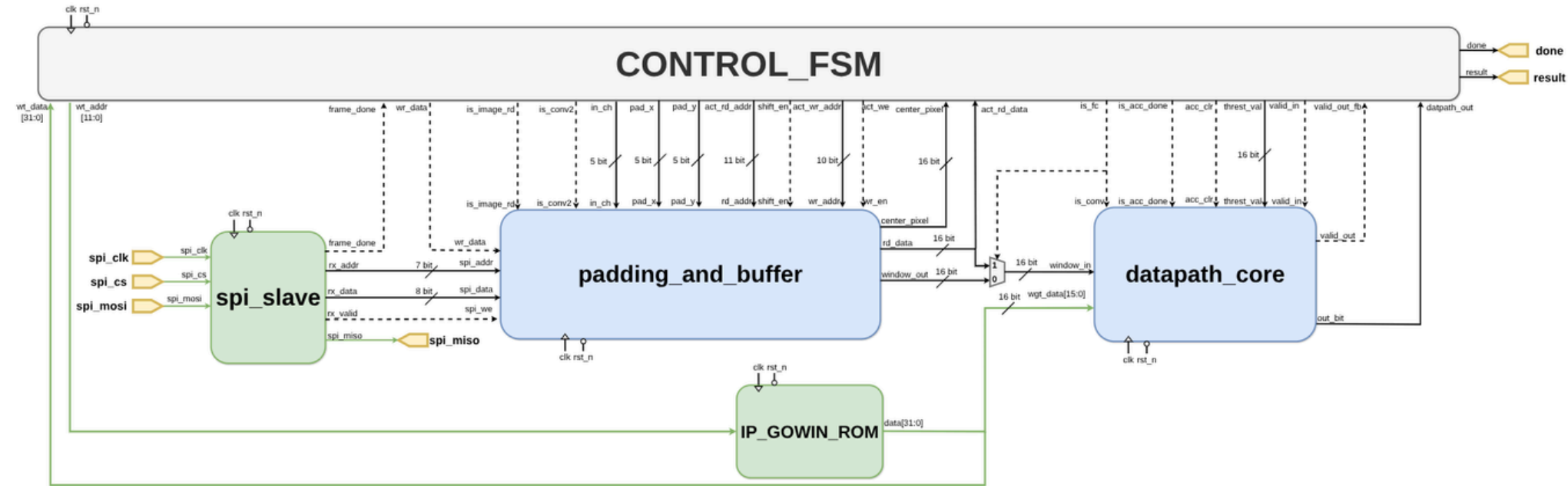
# 3. Kiến trúc hệ thống

- Kiến trúc mạng BNN (FPGA)**



# 3. Kiến trúc hệ thống

## • Kiến trúc RTL mạng BNN



## • Chức năng các module

- + spi\_slave: Module giao tiếp giữa MCU và FPGA
- + padding\_and\_buffer: tạo Line-buffer từ BRAM để tạo sliding window 3x3 và xử lý padding.
- + IP\_GOWIN\_ROM: IP của GOWIN dùng lưu trữ weight và ngưỡng popcount trong BSRAM.
- + datapath\_core: Thay thế hoàn toàn phép tích chập bằng toán tử XNOR và bộ đếm Popcount.
- + CONTROL\_FSM: Điều khiển các state từ nhận ảnh cho đến tính toán từng layer cho đến hết.

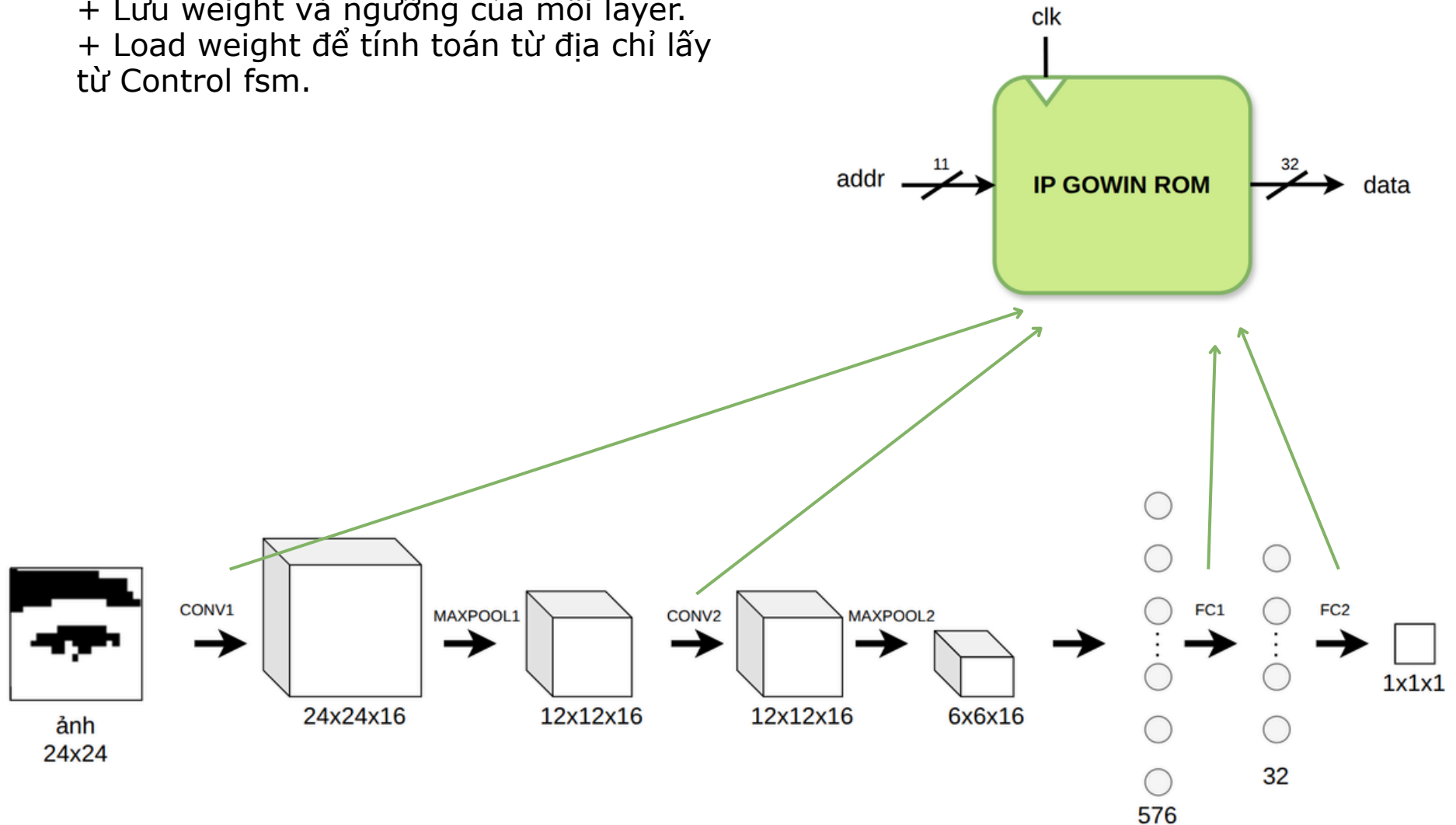
# 3. Kiến trúc hệ thống

## • Module IP\_GOWIN\_ROM

### Chức năng:

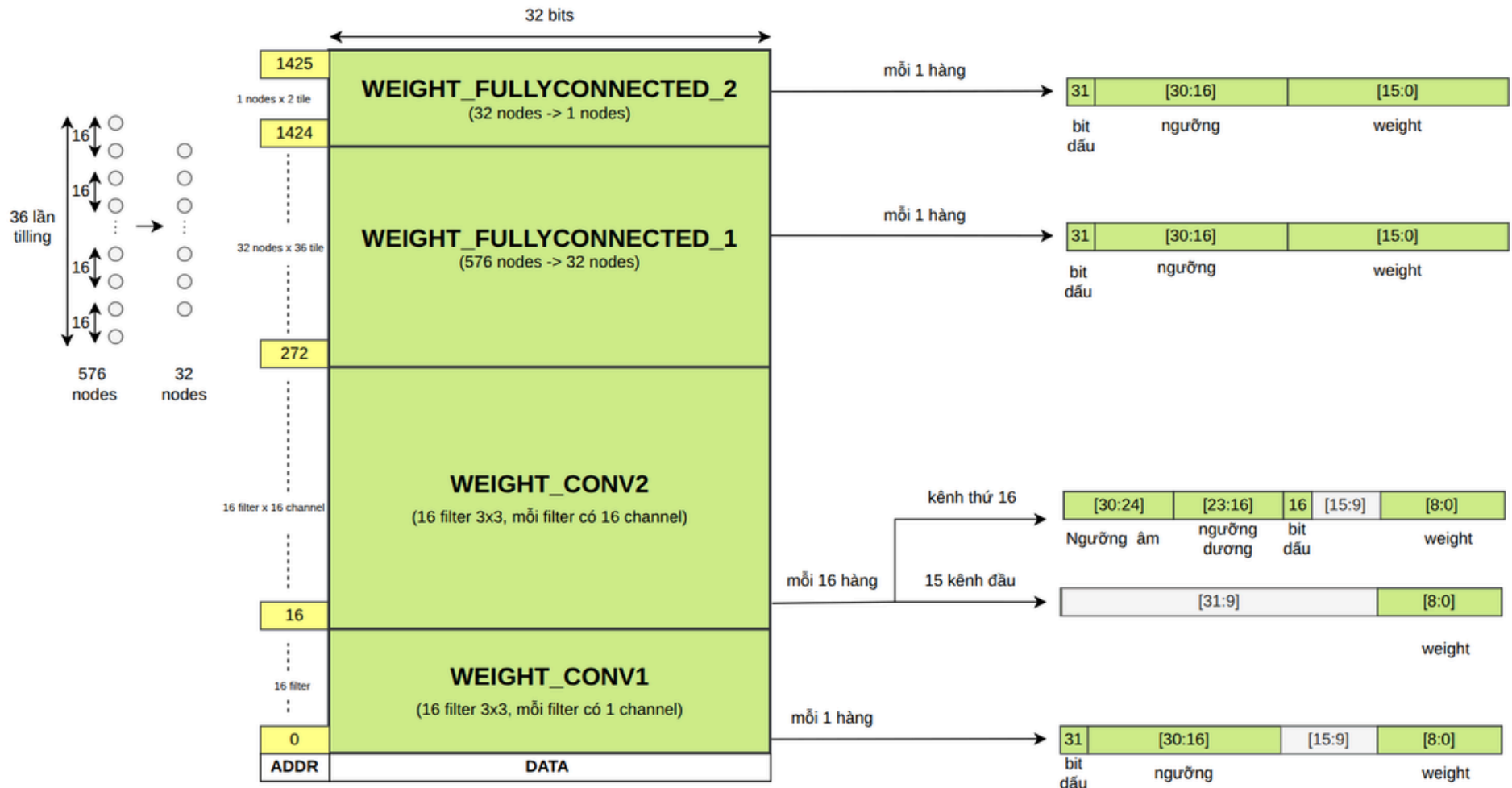
- + Lưu weight và ngưỡng của mỗi layer.
- + Load weight để tính toán từ địa chỉ lấy từ Control fsm.

### Block design:



# 3. Kiến trúc hệ thống

## • Module IP\_GOWIN\_ROM

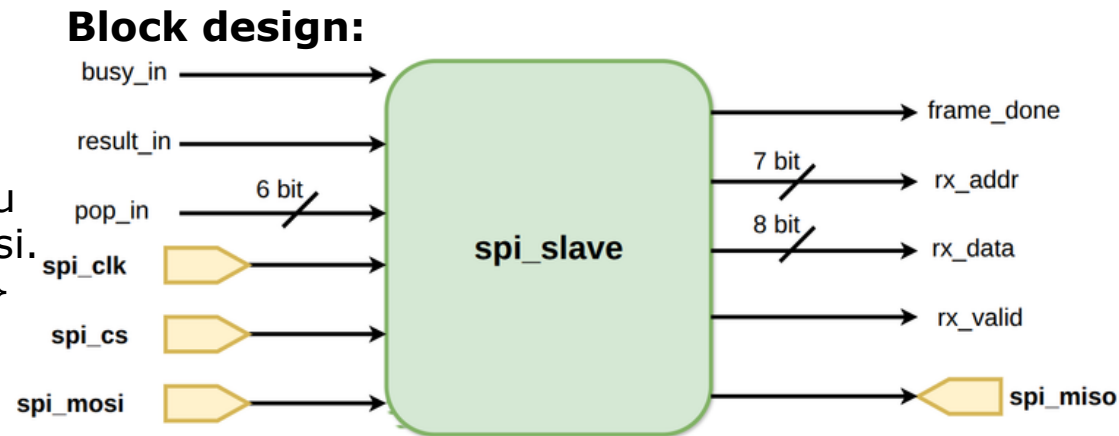


# 3. Kiến trúc hệ thống

## • Module SPI Slave

### Chức năng:

- + Nhận ảnh 24x24 bit (72 byte) từ mcu vào bram của fpga thông qua chân mosi.
- + về 1 byte {busy\_in,pop\_in,result\_in} thông qua chân miso.



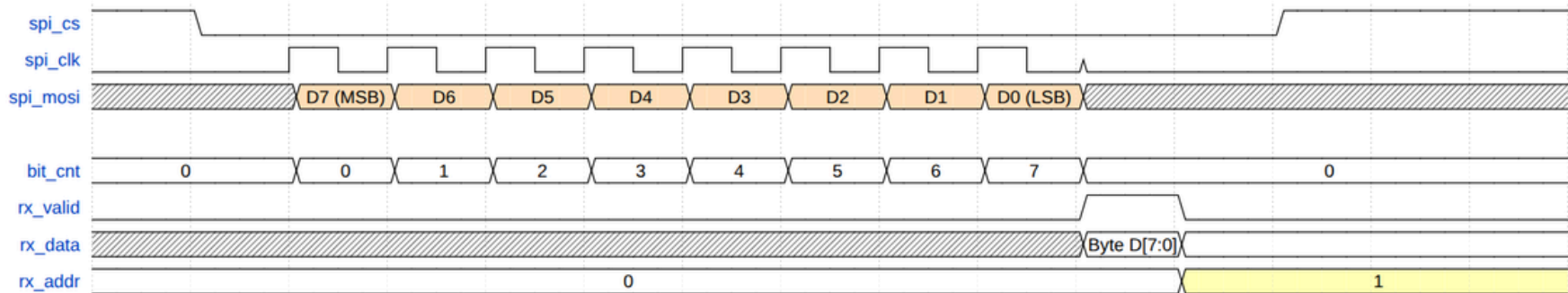
| Port                       | In/Out | bits | Mô tả chức năng (Description)                                                     |
|----------------------------|--------|------|-----------------------------------------------------------------------------------|
| spi_clk                    | Input  | 1    | Xung nhịp SPI (SCK) nhận từ vi điều khiển (MCU Master).                           |
| spi_cs                     | Input  | 1    | Tín hiệu chọn chip (Chip Select/SS) nhận từ MCU.                                  |
| spi_mosi                   | Input  | 1    | Đường dữ liệu MOSI nhận luồng bit ảnh (payload) từ MCU.                           |
| spi_miso                   | Output | 1    | Đường dữ liệu MISO truyền byte trạng thái (busy, popcount, result) về lại MCU.    |
| rx_valid                   | Output | 1    | Xung cờ báo hiệu đã nhận và dịch xong 1 byte hợp lệ từ đường MOSI.                |
| rx_data                    | Output | 8    | Byte dữ liệu ảnh vừa được nhận hoàn chỉnh để ghi vào bộ nhớ RAM (act_ram).        |
| rx_addr                    | Output | 7    | Địa chỉ ghi bộ nhớ tương ứng với byte nhận được (chạy từ 0 đến TOTAL_BYTES - 1).  |
| frame_done                 | Output | 1    | Xung cờ báo hiệu đã nhận đủ 1 khung hình (72 bytes), kích hoạt FSM bắt đầu xử lý. |
| {busy_in,pop_in,result_in} | Input  | 8    | Đưa vào MISO về MCU                                                               |

# 3. Kiến trúc hệ thống

## • Waveform truyền ảnh 24x24 của SPI Slave

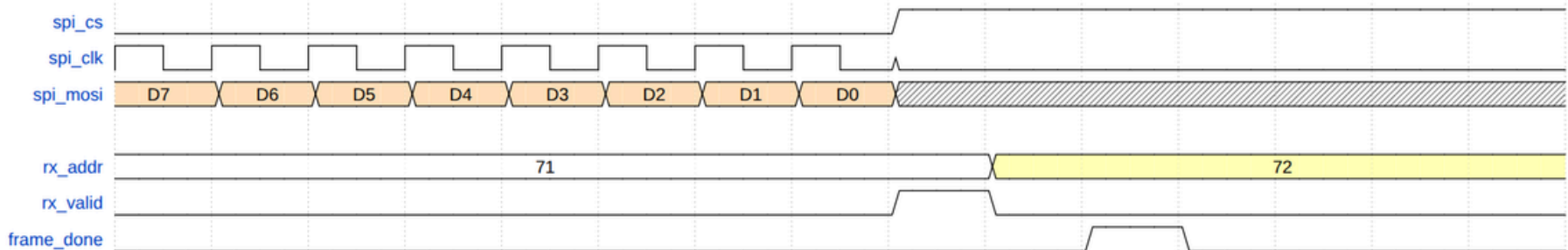
### + Truyền BYTE đầu tiên (0):

Mô tả: MCU kéo CS xuống thấp, truyền lần lượt 8 bit (MSB First). Trong lúc đọc bit D0, **bit\_cnt** mang giá trị 7. Khi đủ 8 bit, FPGA bật cờ **rx\_valid**, chốt dữ liệu, reset **bit\_cnt** về 0 và tự động tăng **rx\_addr** ở nhịp tiếp theo.



### + Truyền BYTE cuối cùng (71):

Mô tả: MCU truyền xong byte cuối cùng (tại địa chỉ 71) và kéo CS lên cao. FPGA chốt byte, **rx\_addr** tăng lên 72. Hệ thống phát hiện sườn lên của CS và bật cờ **frame\_done** để kích hoạt FSM.



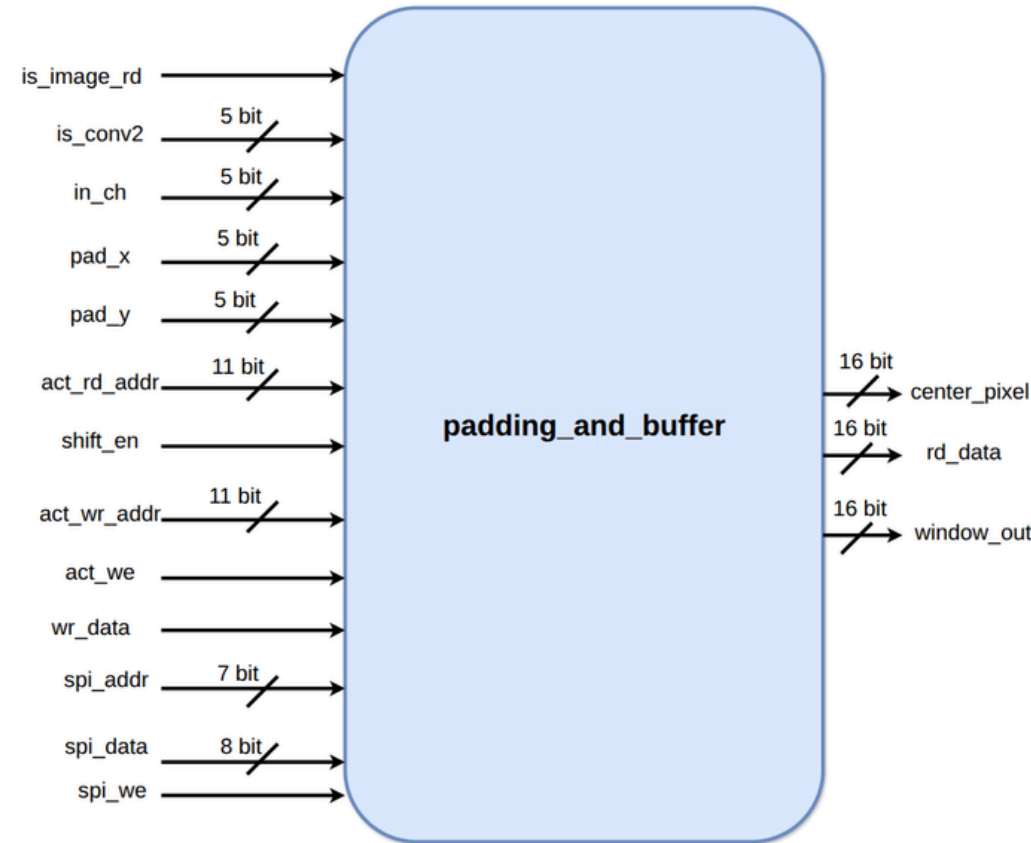
# 3. Kiến trúc hệ thống

## • Module Padding\_and\_buffer

### Chức năng:

- + Dùng bram để chứa output các layer
- + Tạo slidding window 3x3 cho conv1 và conv2 bằng phương pháp dùng Line Buffer để đẩy vào core tính tích chập XNOR.
- + Thêm Padding cho conv1 và conv2.

### Block design:



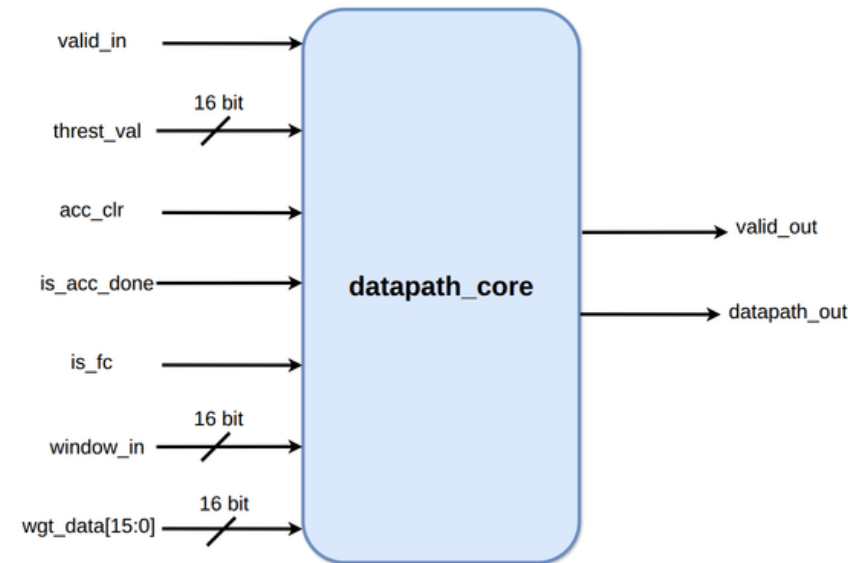


# 3. Kiến trúc hệ thống

## • Module Datapath\_core

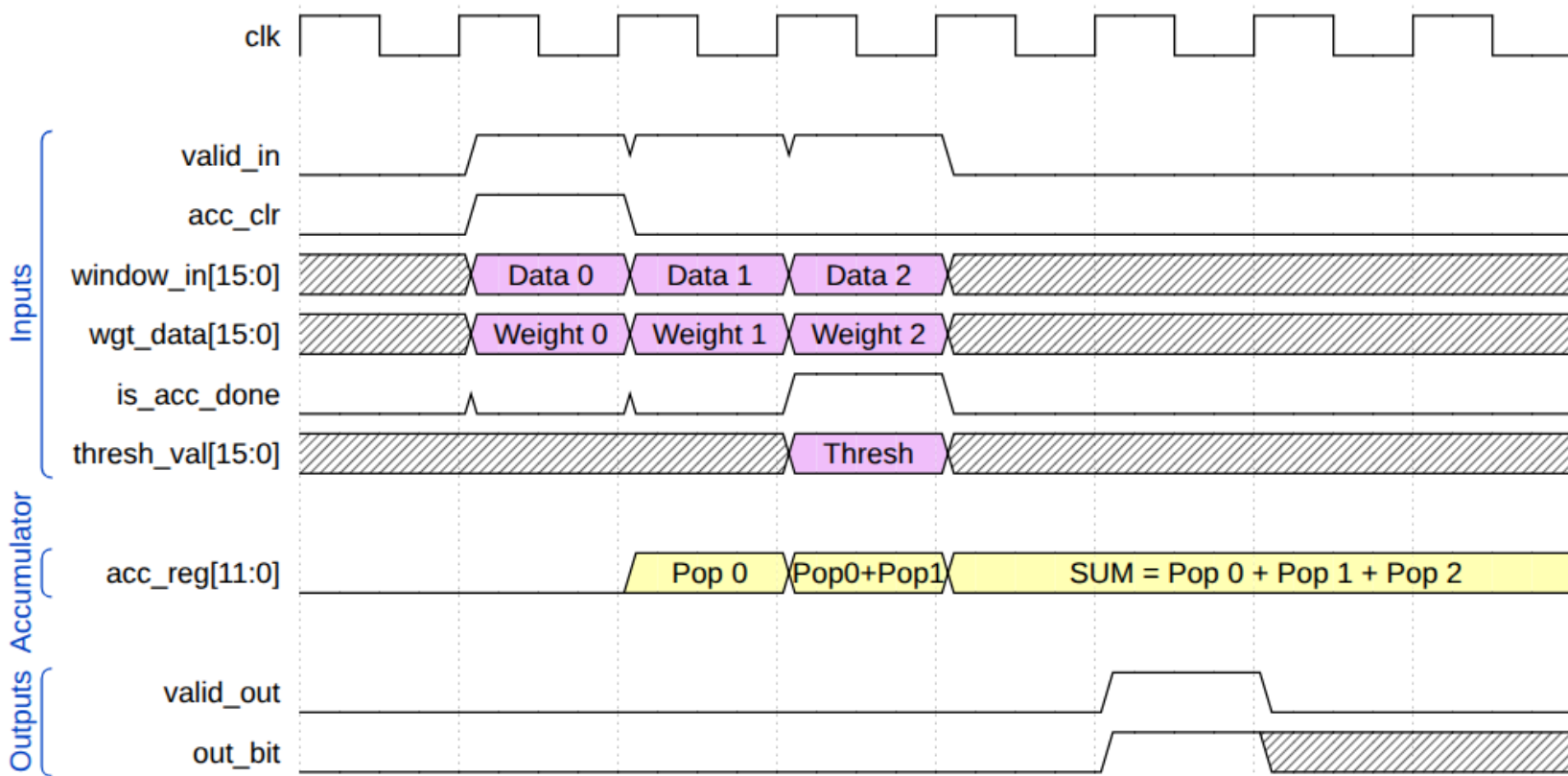
- + **Phép nhân XNOR**: Thực hiện "nhân" nhị phân giữa dữ liệu đầu vào và trọng số bằng cổng XNOR.
- + **Bộ đếm Popcount**: Đếm tổng số lượng bit 1 sinh ra từ kết quả của phép toán XNOR.
- + **Bộ cộng dồn**: Tích lũy liên tục kết quả **Popcount** qua nhiều chu kỳ xung nhịp.
- + **Thresholding**: So sánh tổng giá trị tích lũy với mức ngưỡng để chốt đầu ra là 0 hay 1.

### Block design:



# 3. Kiến trúc hệ thống

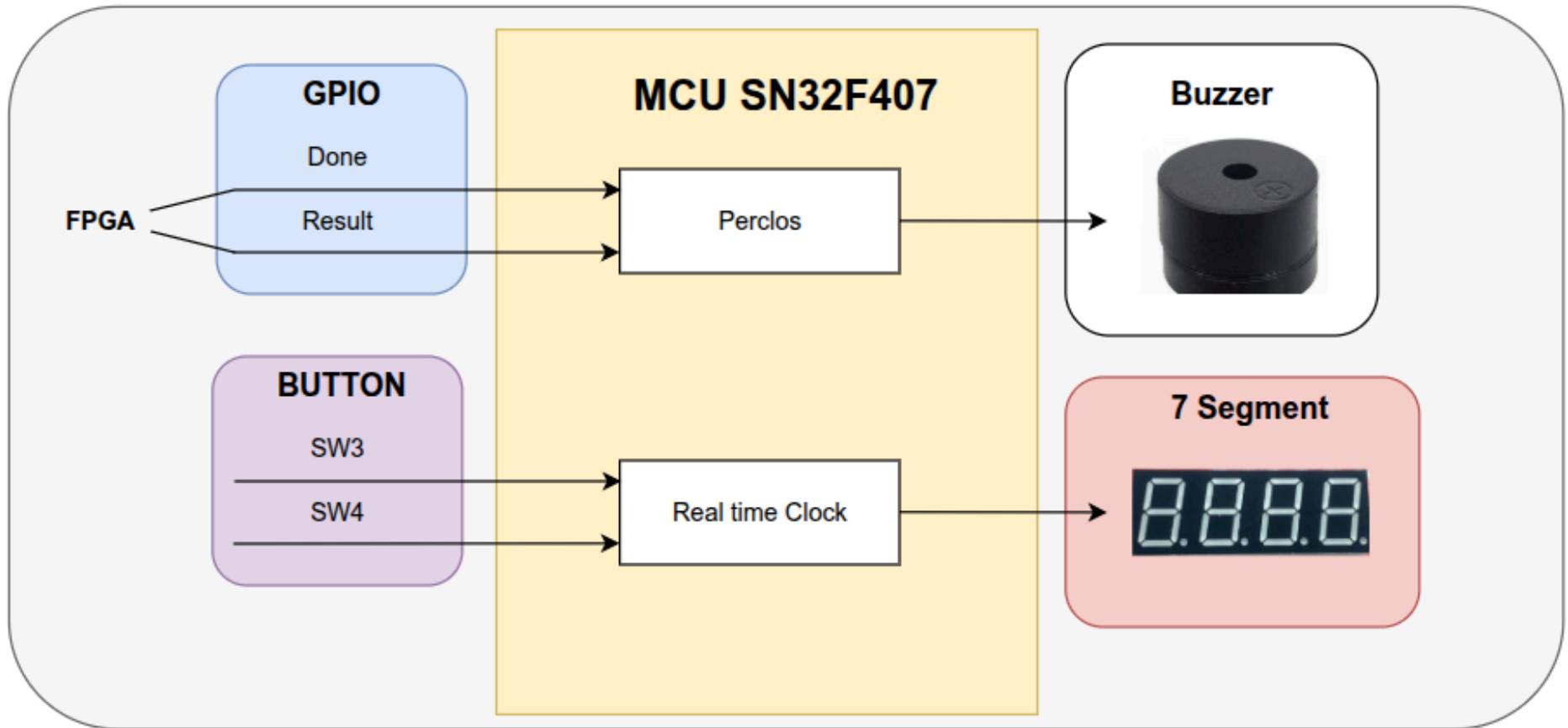
## • Module Datapath\_core



- Pop (Popcount): đếm số lượng 1 trong kết quả XNOR giữa ifm data và weight.
- SUM (acc\_reg): Bộ cộng dồn tích lũy các giá trị Pop qua từng nhịp xung nhịp.
- Thresh (thresh\_val): Ngưỡng kích hoạt . So sánh SUM với Thresh để quyết định ra kết quả 0 hay 1 (out\_bit).

### 3. Kiến trúc hệ thống

- Perclos & Real time Clock(FPGA - MCU Sonix Sn32F407)



### 3. Kiến trúc hệ thống

| Unit         | File                     | Register | LUT      | ALU    | BSRAM | SSRAM |
|--------------|--------------------------|----------|----------|--------|-------|-------|
| ▼ top        | src\top.v                | 1087(0)  | 826(0)   | 112(0) | 4(0)  | 0 (0) |
| spi_slav...  | src\spi_slave.v          | 43(43)   | 33(33)   | 0 (0)  | 0 (0) | 0 (0) |
| ▼ weight_... | src\weight_rom.v         | 1(0)     | 34(0)    | 0 (0)  | 3(0)  | 0 (0) |
| gow...       | src\gowin_rom.v          | 1(1)     | 34(34)   | 0 (0)  | 3(3)  | 0 (0) |
| padding...   | src\padding_and_buffer.v | 820(820) | 173(173) | 0 (0)  | 1(1)  | 0 (0) |
| datapath...  | src\datapath_core.v      | 32(32)   | 87(87)   | 35(35) | 0 (0) | 0 (0) |
| control_...  | src\control_fsm.v        | 191(191) | 499(499) | 77(77) | 0 (0) | 0 (0) |

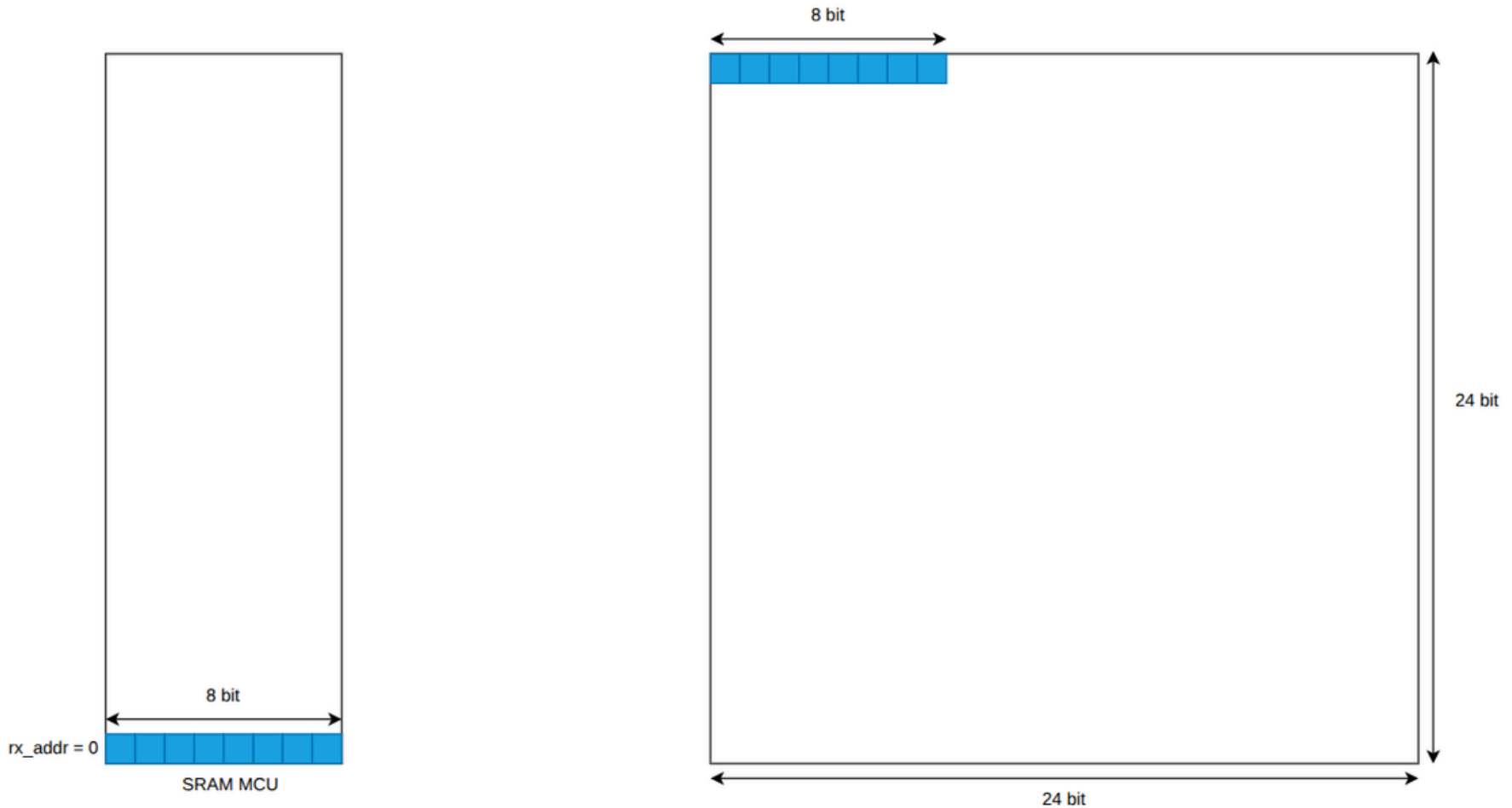


# ***DATAFLOW***



# Lưu ảnh 24x24 vào sram MCU

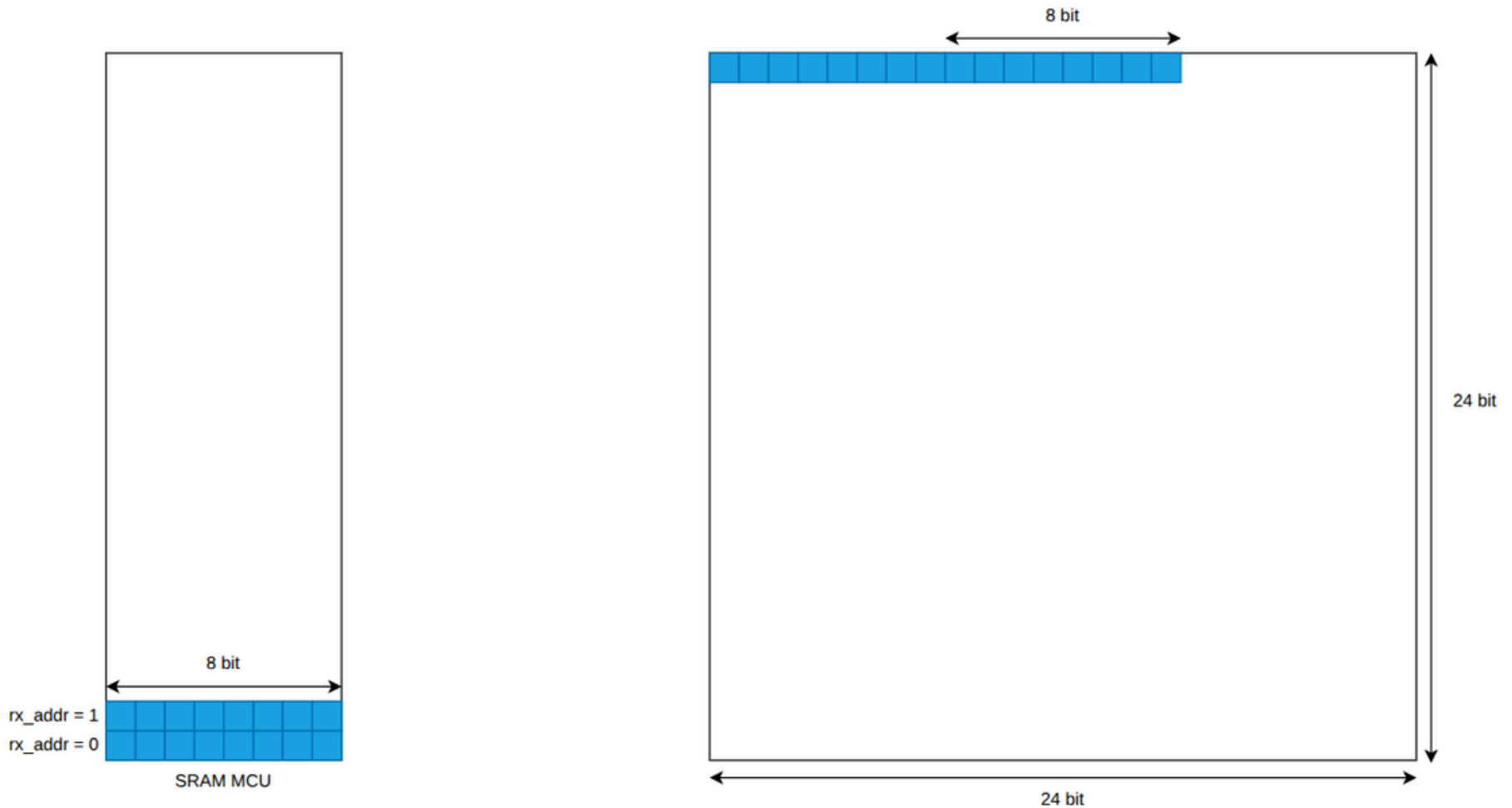
- `uint8_t image_buffer[72];` => mỗi ô nhớ chứa 1 byte (72 = 24x24/16)



ảnh input 24x24

# Lưu ảnh 24x24 vào sram MCU

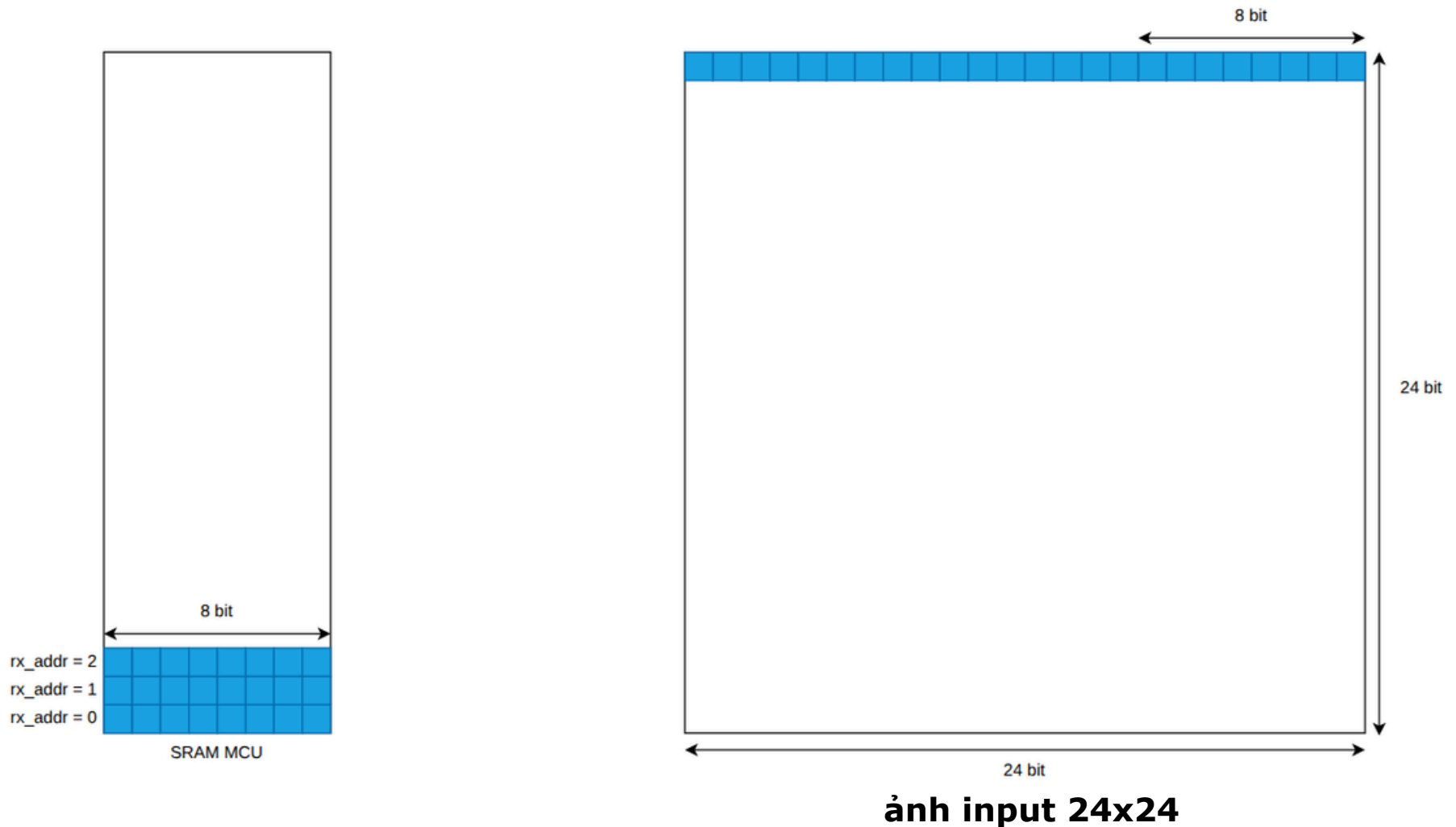
- `uint8_t image_buffer[72];` => mỗi ô nhớ chứa 1 byte



**ảnh input 24x24**

# Lưu ảnh 24x24 vào sram MCU

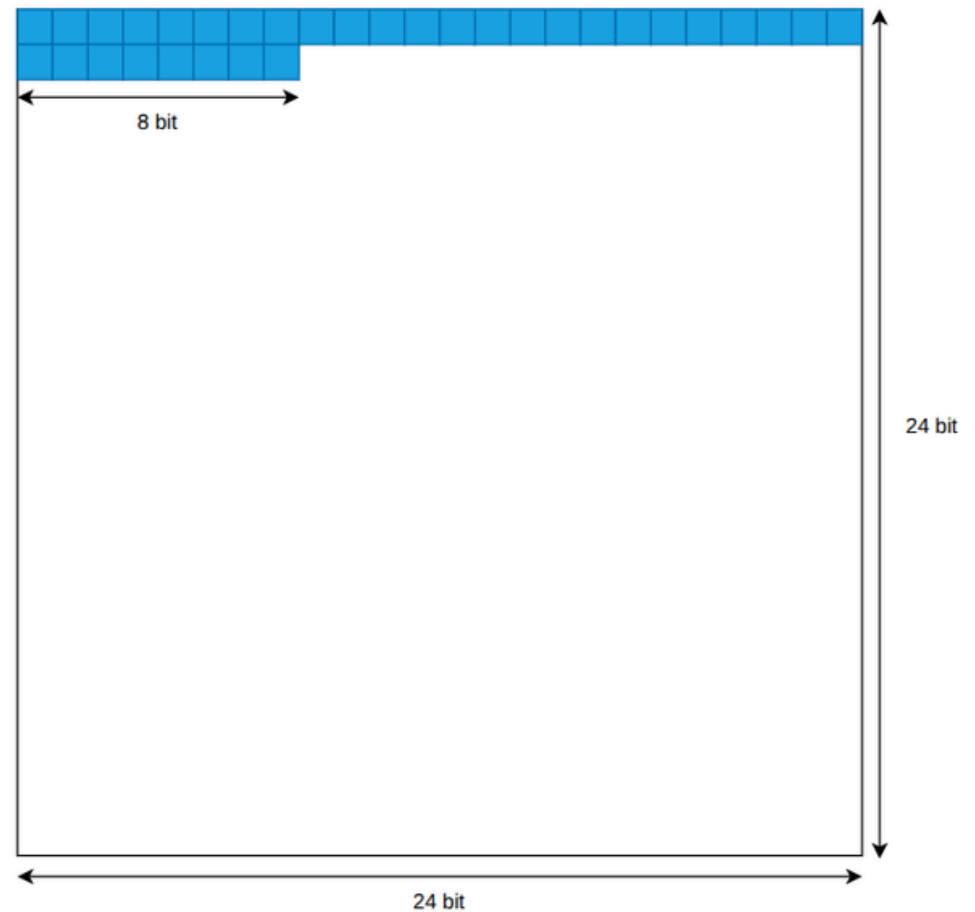
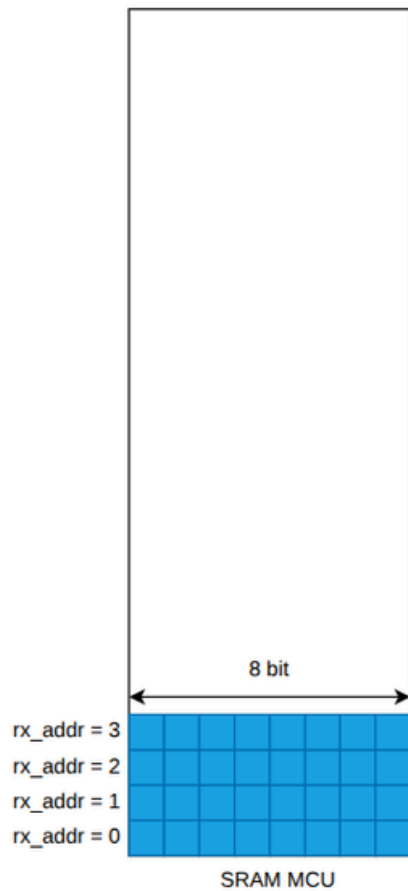
- `uint8_t image_buffer[72];` => mỗi ô nhớ chứa 1 byte





# Lưu ảnh 24x24 vào sram MCU

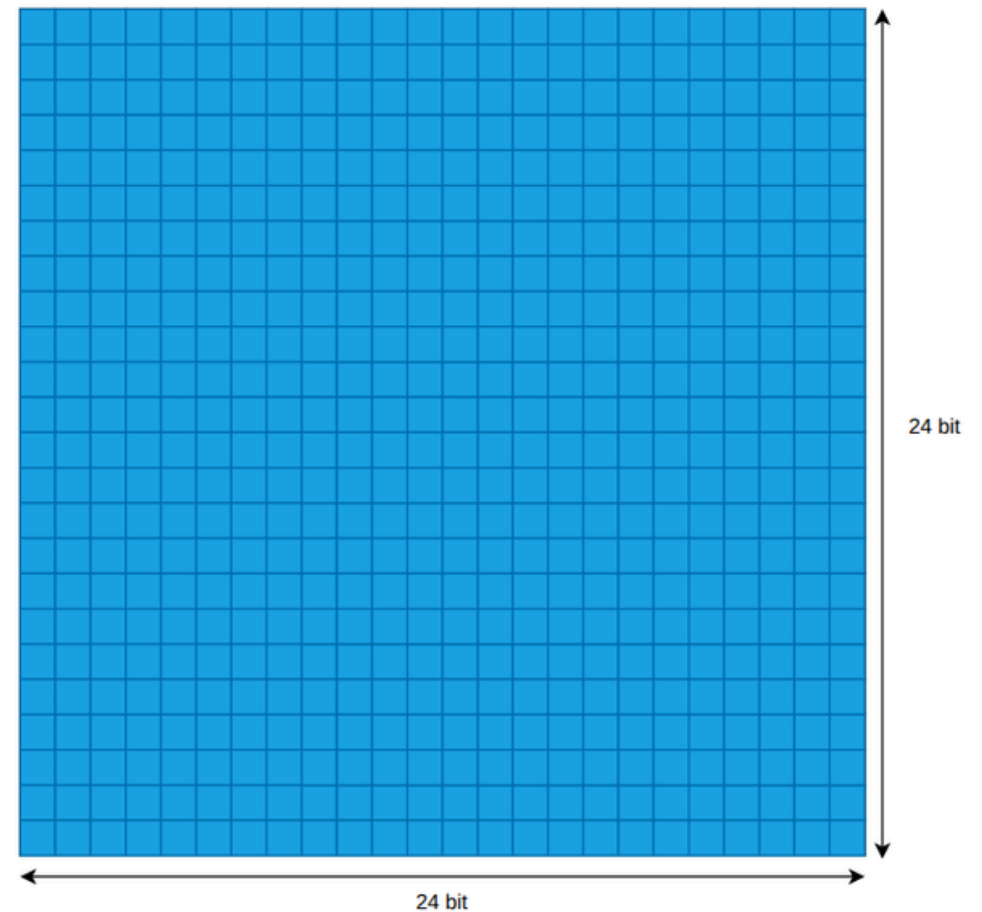
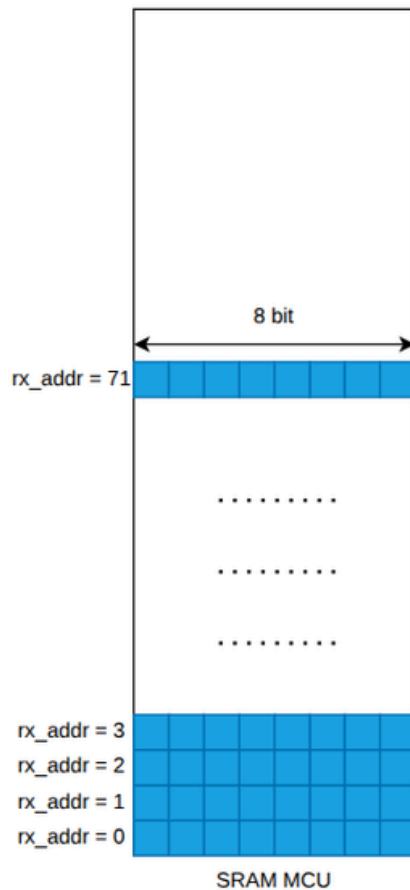
- `uint8_t image_buffer[72];` => mỗi ô nhớ chứa 1 byte



**ảnh input 24x24**

# Lưu ảnh 24x24 vào sram MCU

- `uint8_t image_buffer[72];` => mỗi ô nhớ chứa 1 byte



**ảnh input 24x24**

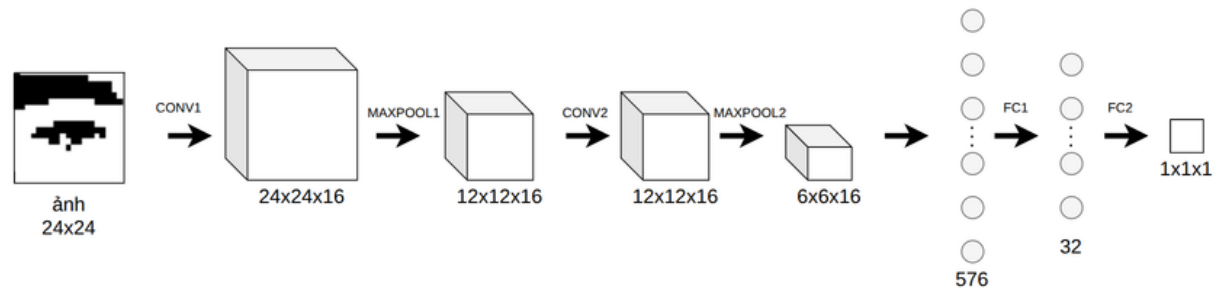
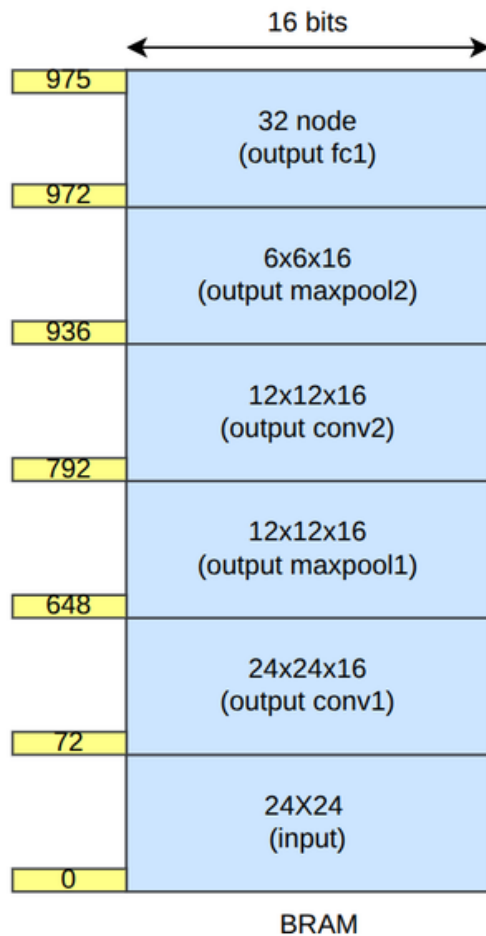


# *Memory Map FPGA*

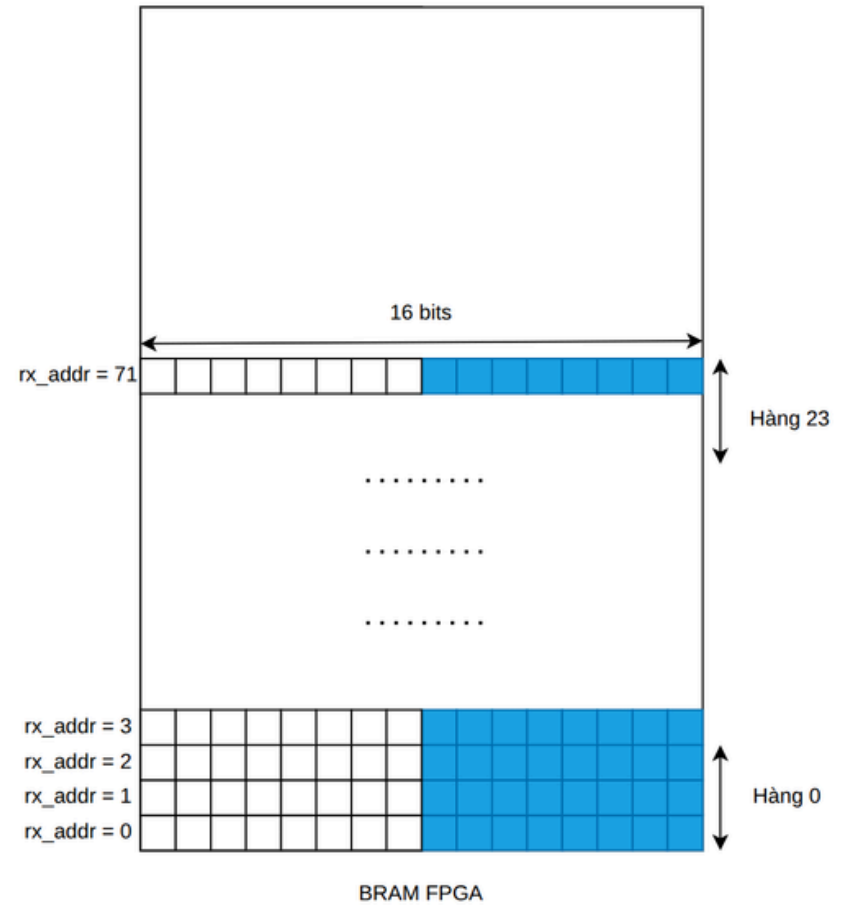
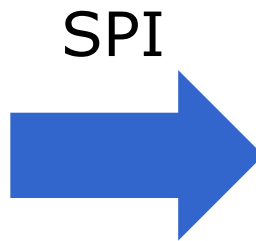
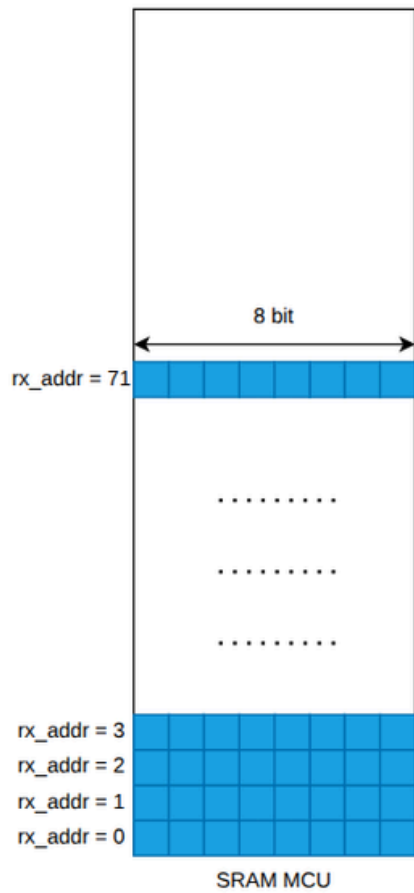


# Memory Map ở FPGA

- Đầu tiên ra khởi tạo 1 BRAM với mục đích lưu toàn bộ output từng layer phục vụ cho việc load ifm và store ofm



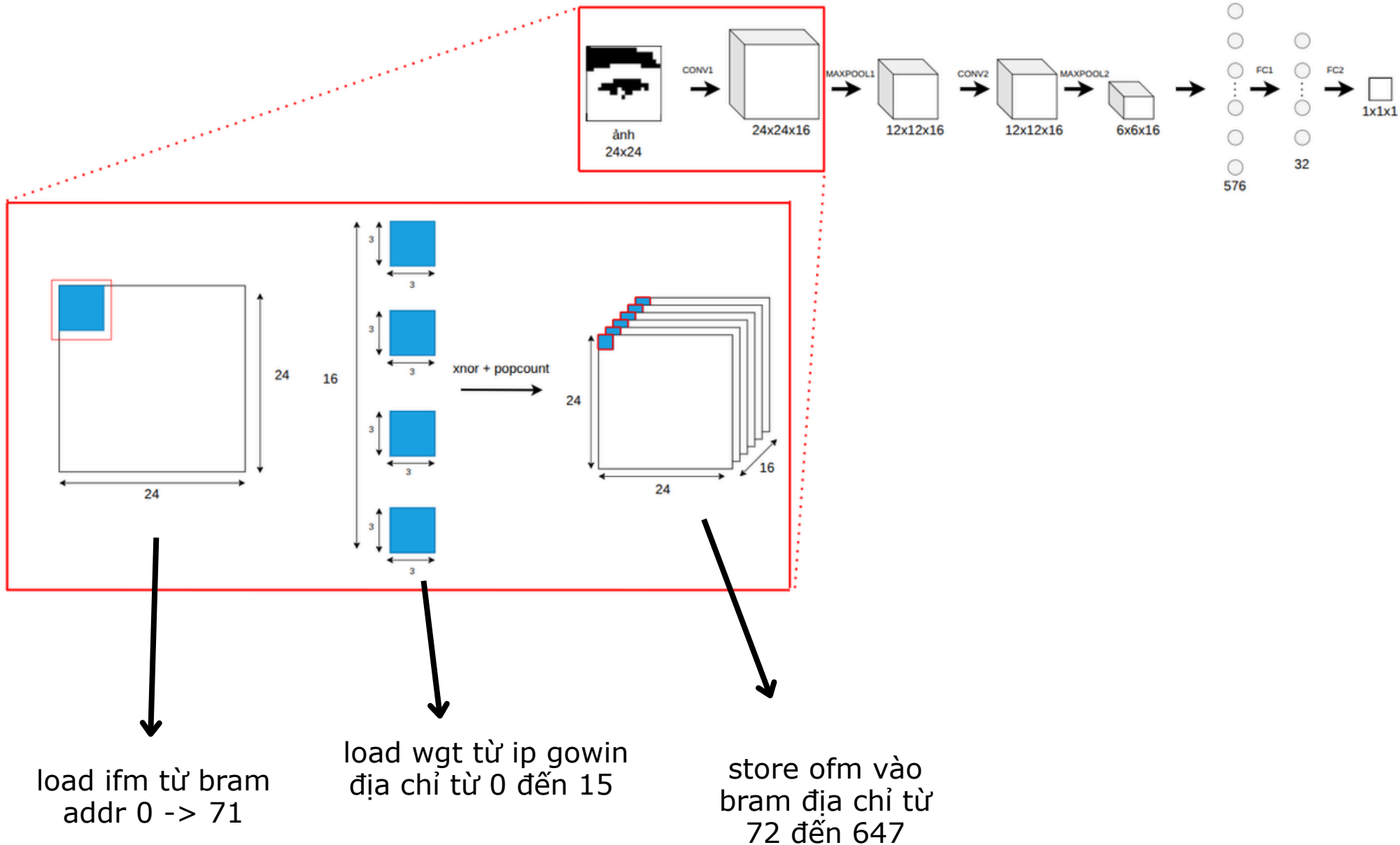
# Truyền input 24x24 từ sram mcu sang bram

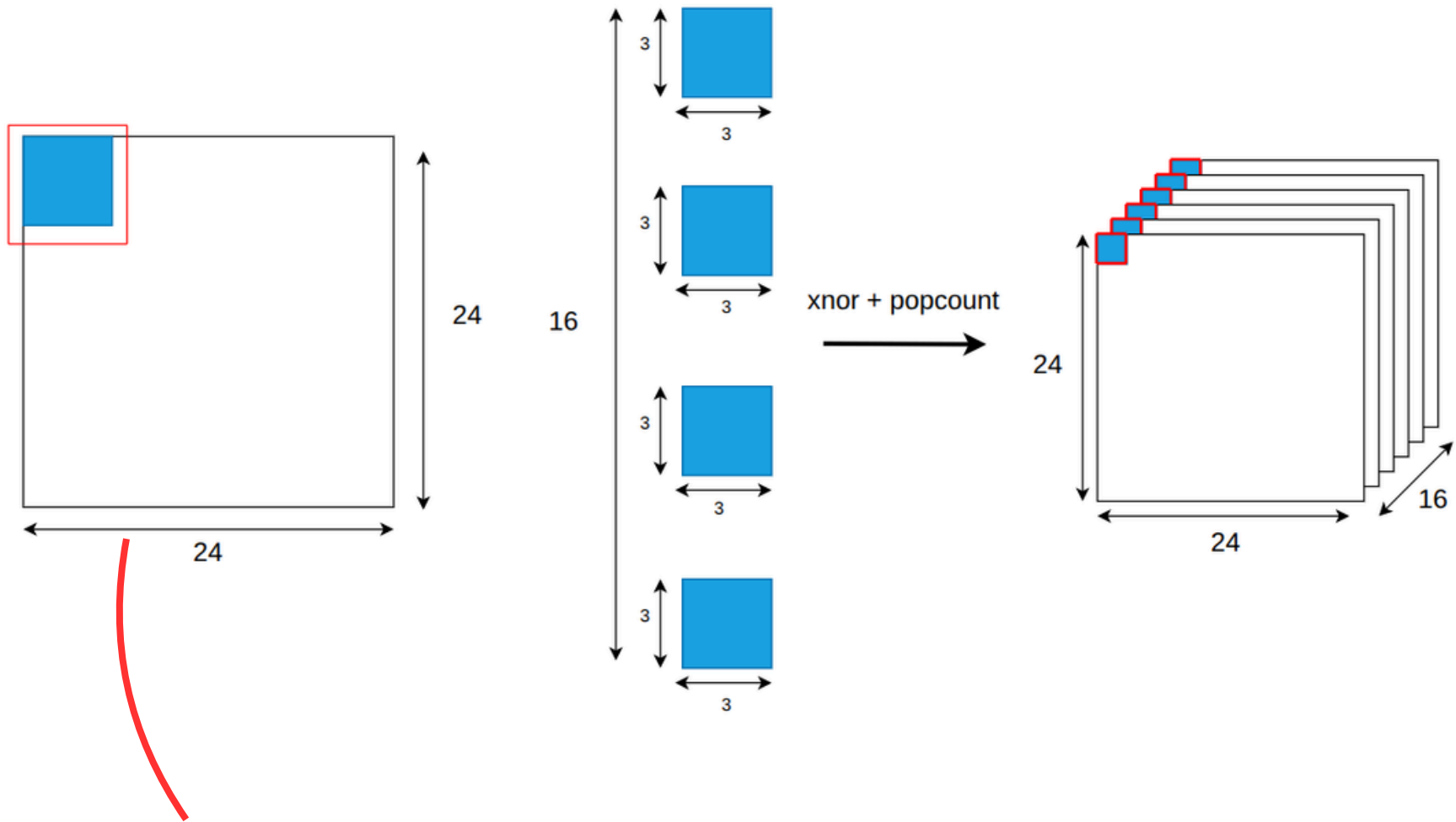




***CONV1***







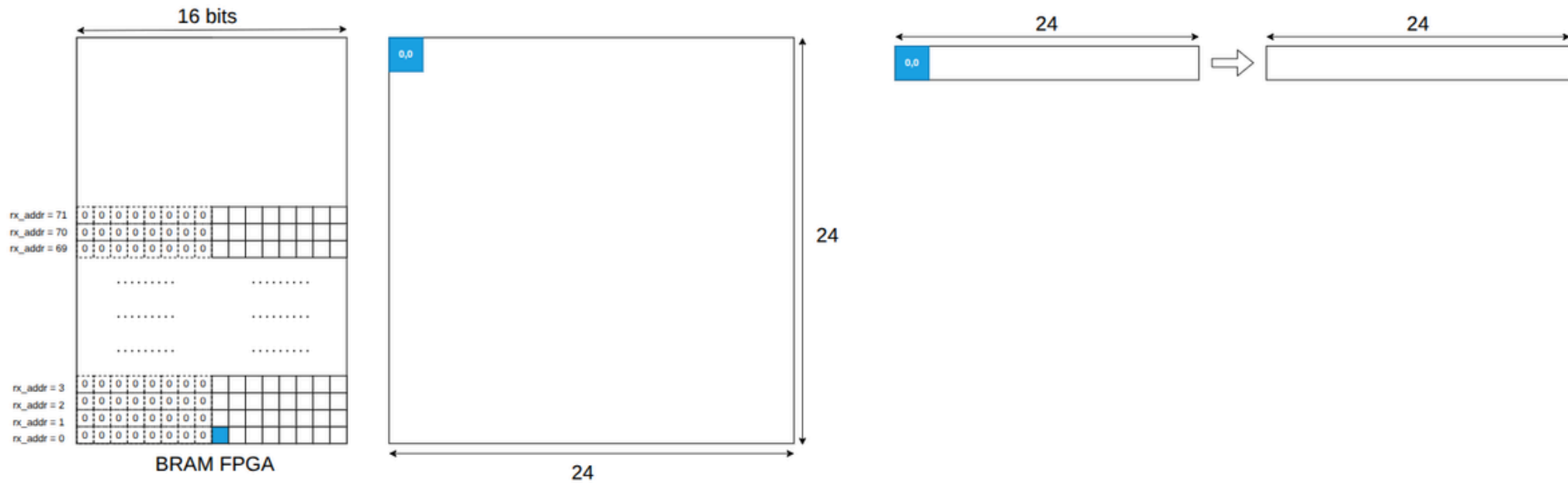
=> Dùng kĩ thuật "Line Buffer" để load window 3x3 ifm từ bram

Ưu điểm:

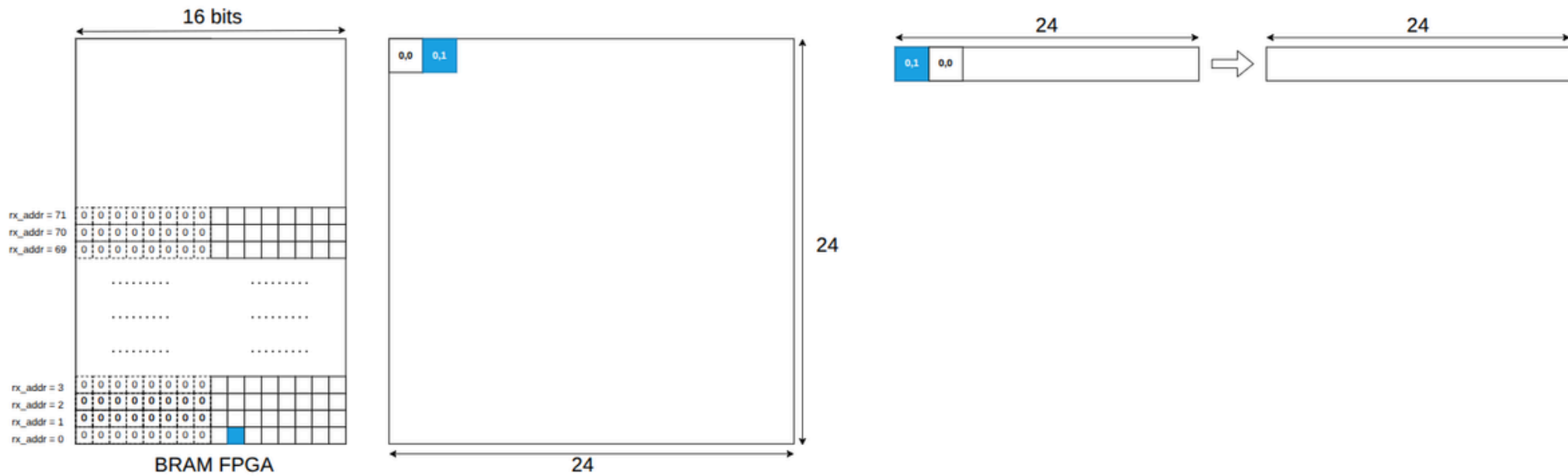
- + Code đơn giản, ít tài nguyên.
- + Tái sử dụng ifm (dùng lại được 2x3 bit mà ko phải load lại từ bram).



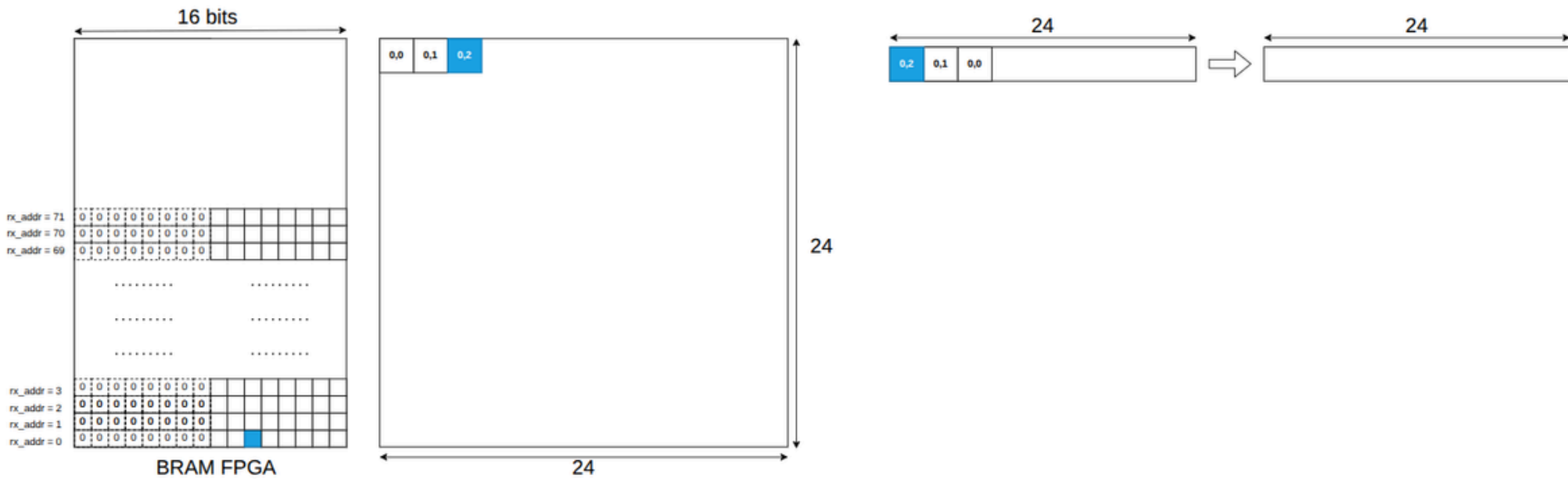
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



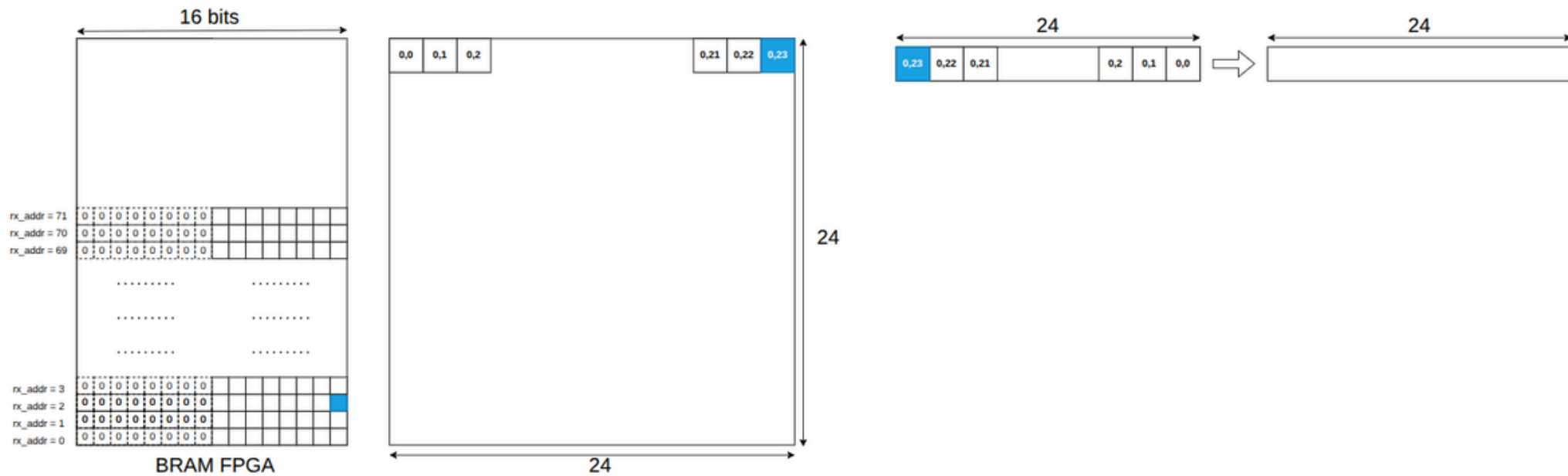
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



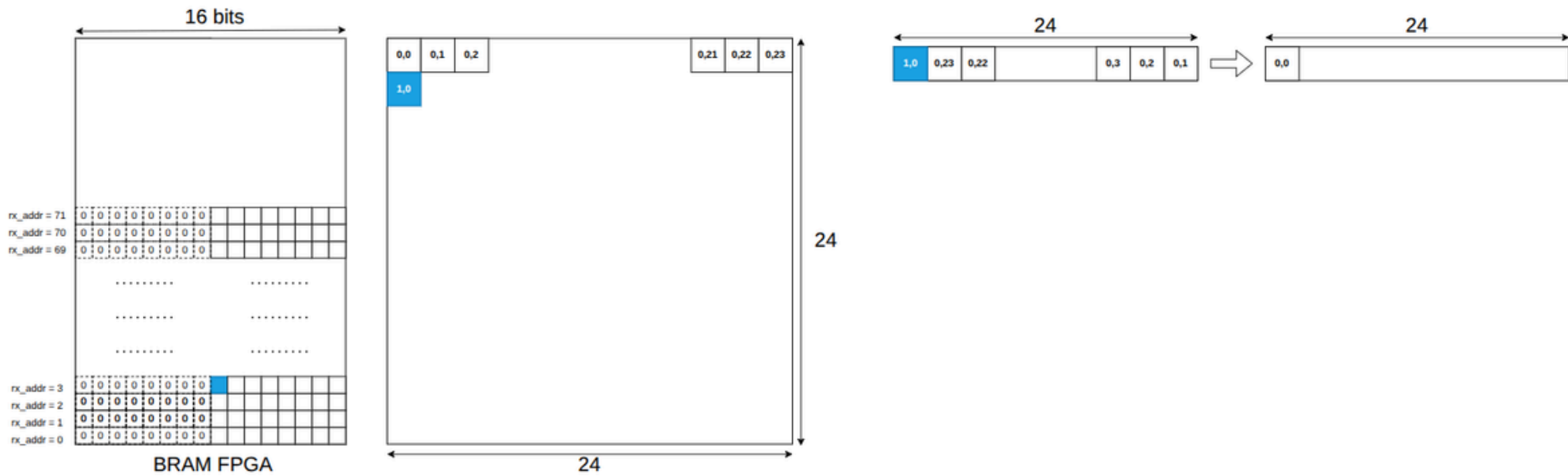
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



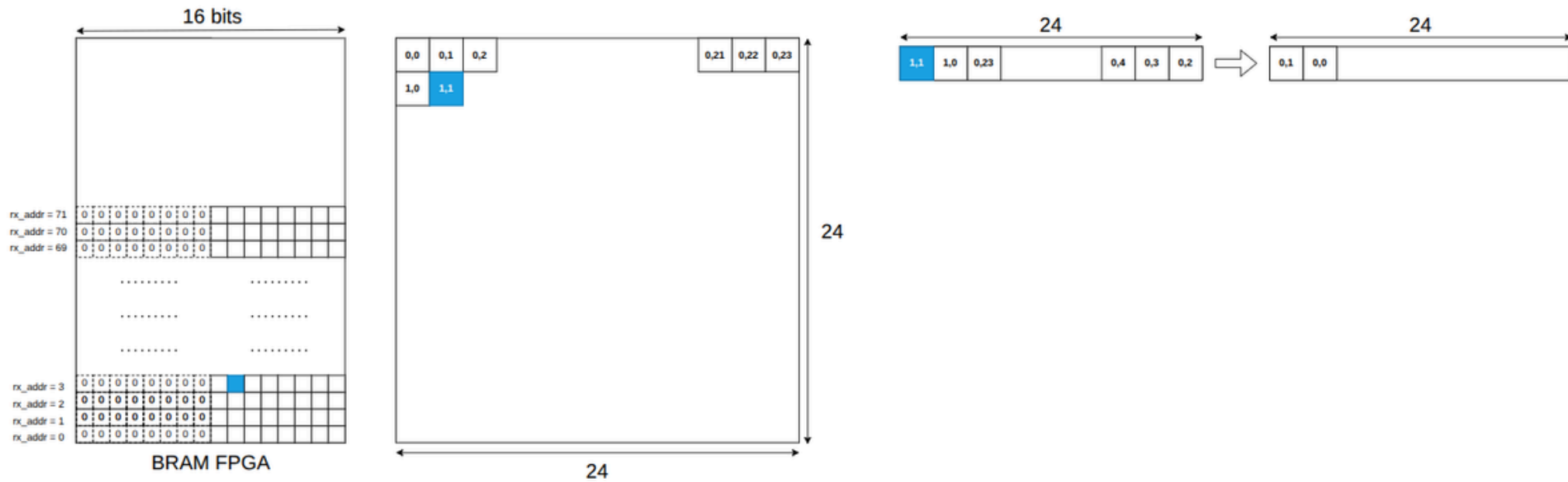
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



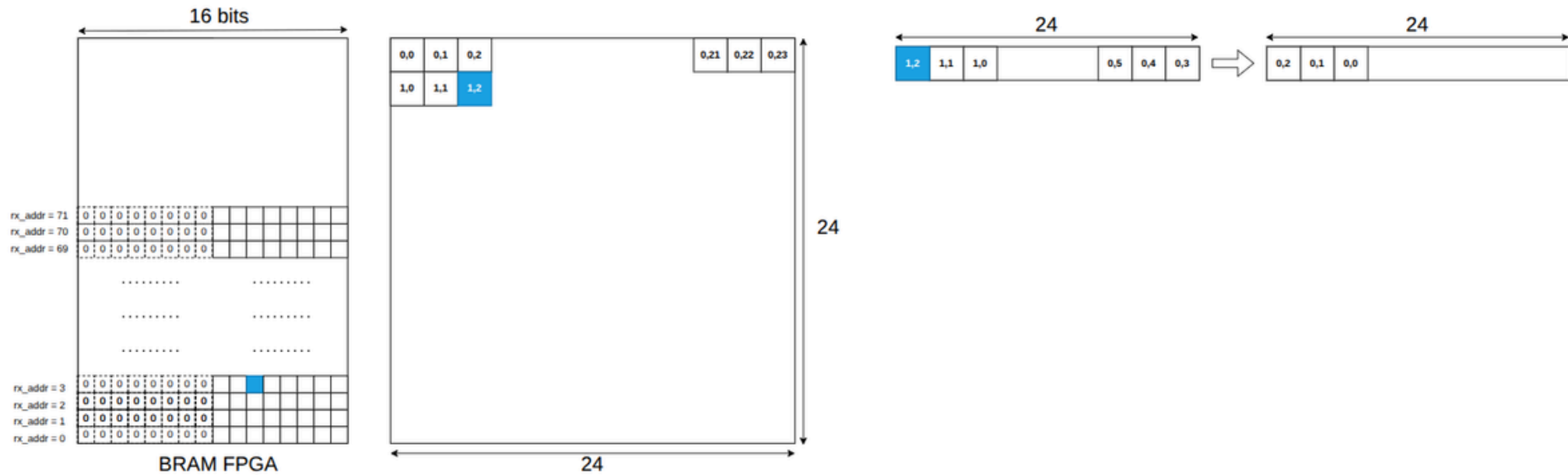
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



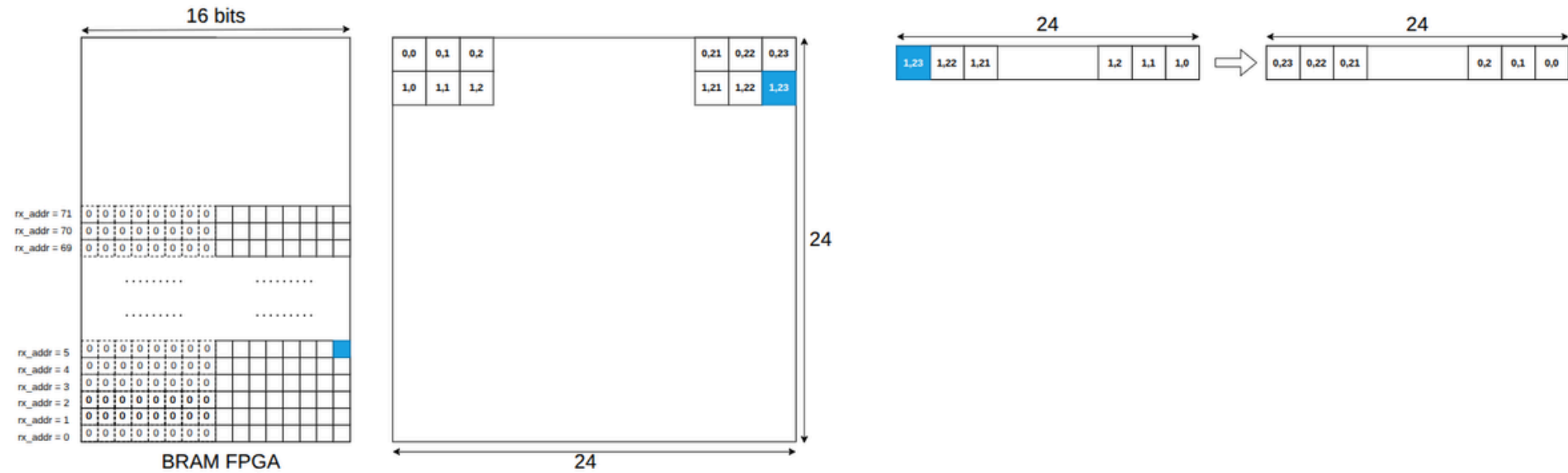
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:

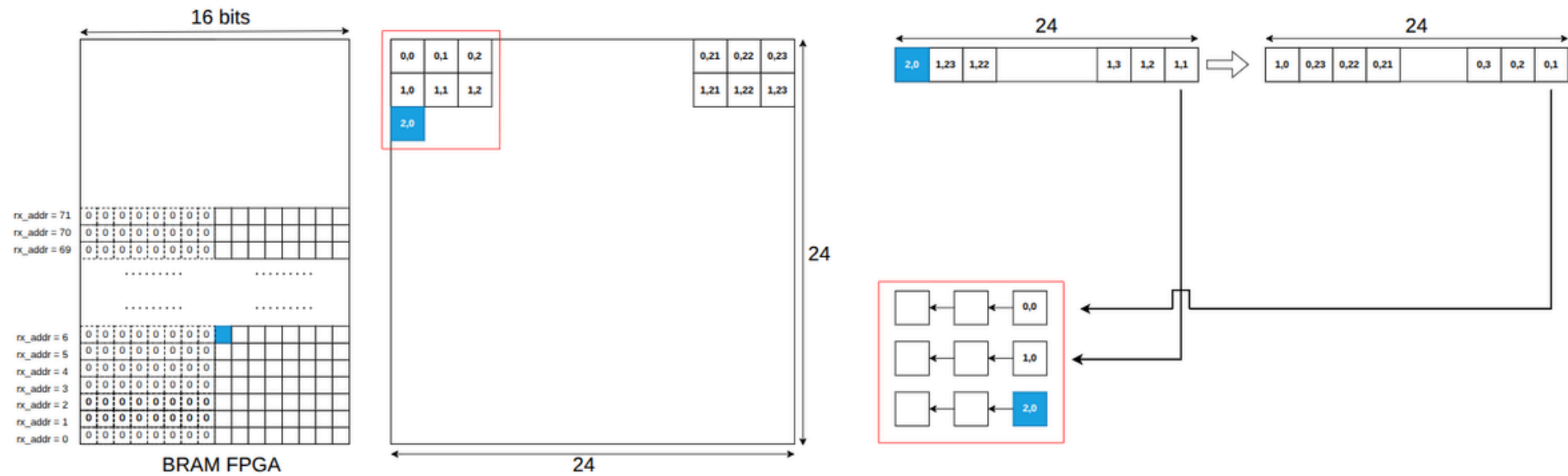


- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:

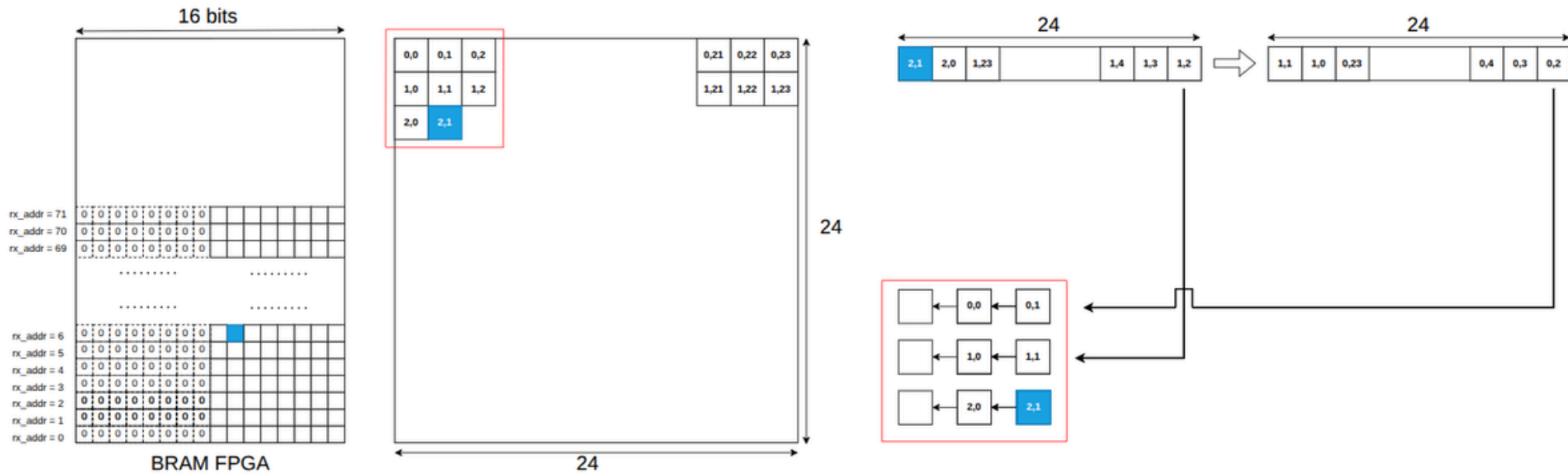




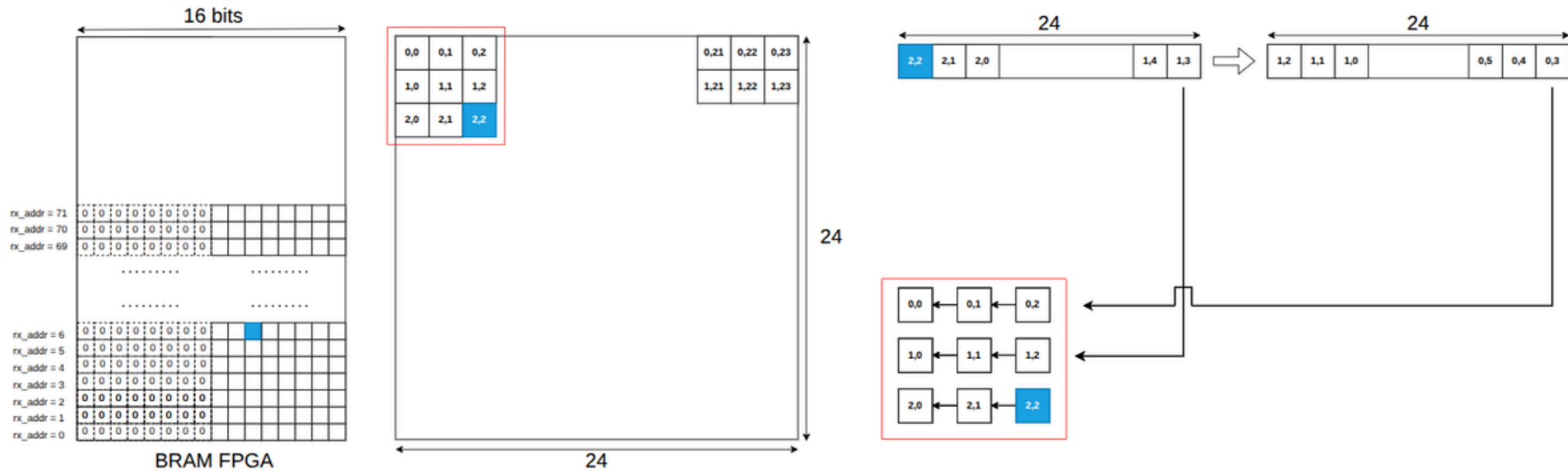
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



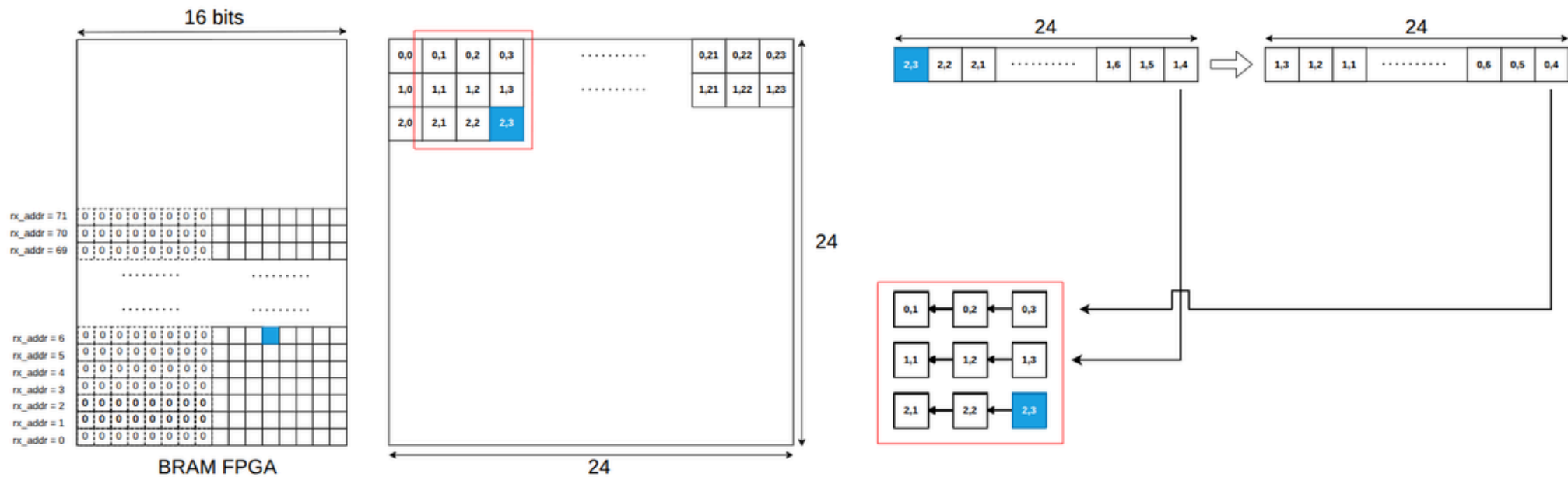
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



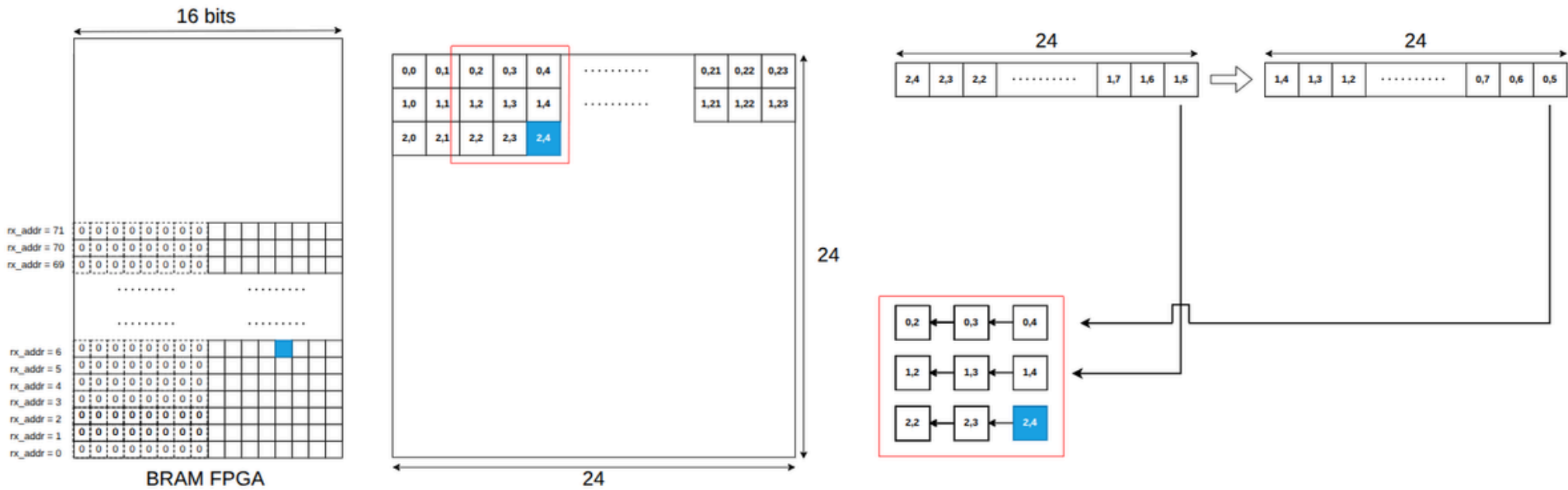
- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:

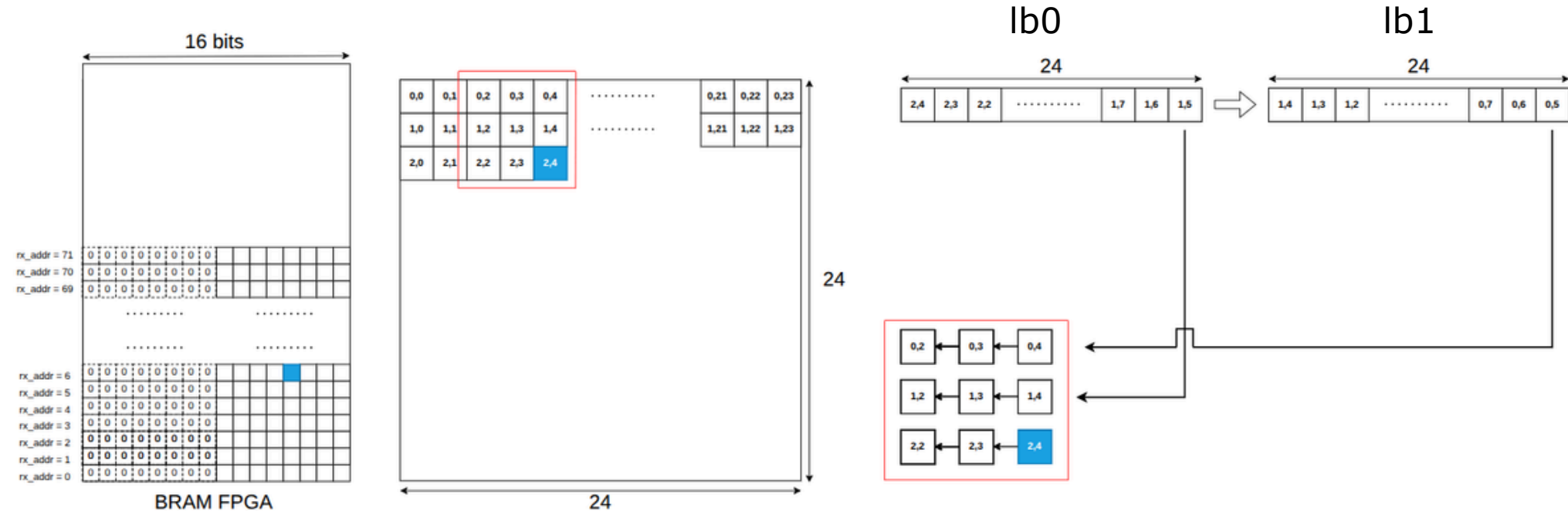


- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:



- Load ifm từ bram rồi tạo cửa sổ 3x3 dùng line buffer:





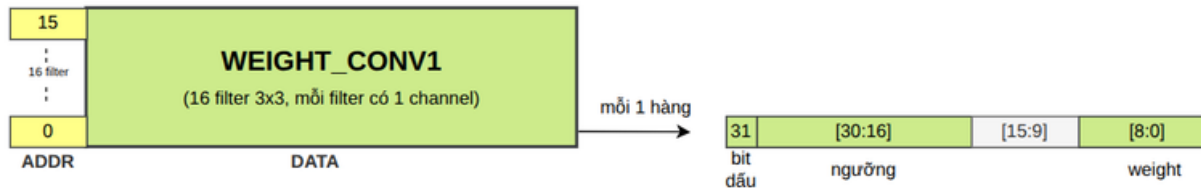
## • Công thức load từng bit từ bram:

```
for(i = 0; i < 24x24; i++){
    addr = i >> 3; // chia 8
    bit_index = i & 0x111; // chia dư cho 8
    data = (bram[addr] >> (7 - bit_index)) & 0x1;
}
```

## • Công thức line buffer để tạo window 3x3

```
lb0[24] = {0}, lb1[24] = {0};
w00=0,w01=0,w02=0,w10=0,w11=0,w12=0,w20=0,w21=0,w22=0;
w00 = w01; w01 = w02; w02 = lb1[11];
w10 = w11; w11 = w12; w12 = lb0[11];
w20 = w21; w21 = w22; w22 = data;
for (int i = 23; i > 0; i--) {
    lb0[i] = lb0[i-1];
    lb1[i] = lb1[i-1];
}
lb0[0] = data;
lb1[0] = lb0[23];
```

- Load wgt và ngưỡng của conv1 từ ip gowin rom:



- Công thức load từ gowin\_rom:

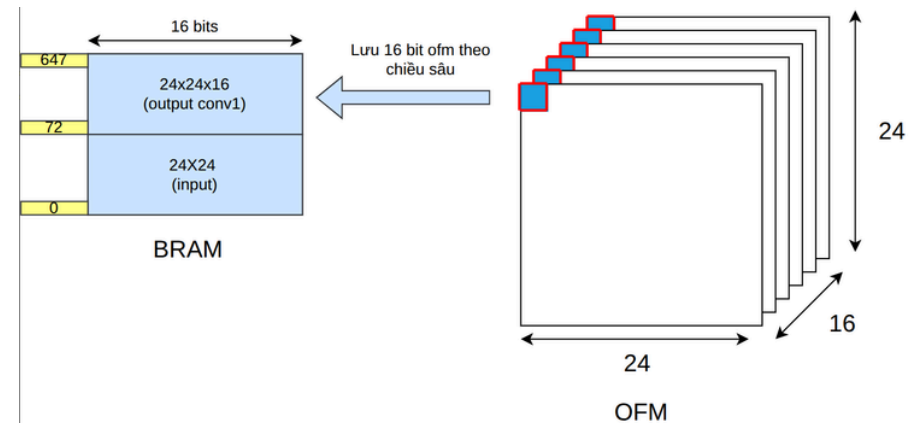
```
for(i = 0; i < 16; i++){
    wgt = gowin_rom[i] & 0x9;
    ngưỡng = (gowin_rom[i] >> 16) & 0xFFFF;
    bit_dấu = (gowin_rom[i] >> 31) & 0x1;
}
```

- **compute (xnor & popcount) và store ofm**

```
uint16 ifm = 0;
ifm |= ((pad_top || pad_left) ? 0 : w00) << 8;
ifm |= ((pad_top) ? 0 : w01) << 7;
ifm |= ((pad_top || pad_right) ? 0 : w02) << 6;
ifm |= ((pad_left) ? 0 : w10) << 5;
ifm |= ( w11) << 4;
ifm |= ((pad_right) ? 0 : w12) << 3;
ifm |= ((pad_bottom || pad_left) ? 0 : w20) << 2;
ifm |= ((pad_bottom) ? 0 : w21) << 1;
ifm |= ((pad_bottom || pad_right) ? 0 : w22) << 0;

for(channel = 0; channel < 16; channel ++){
    xnor = ~(ifm ^ wgt) & 0x01FF;
    int pop_count = 0;
    for(int i = 0; i < 9; i++) {
        pop_count += (xnor >> i) & 0x1;
    }
    if(bit_dấu == 1) output = (pop_count >= ngưỡng);
    else          output = (pop_count < ngưỡng);

    shift_reg = (output << 15) | (shift_reg >> 1);
}
addr = 72;
Bram [addr++] = shift_reg;
```

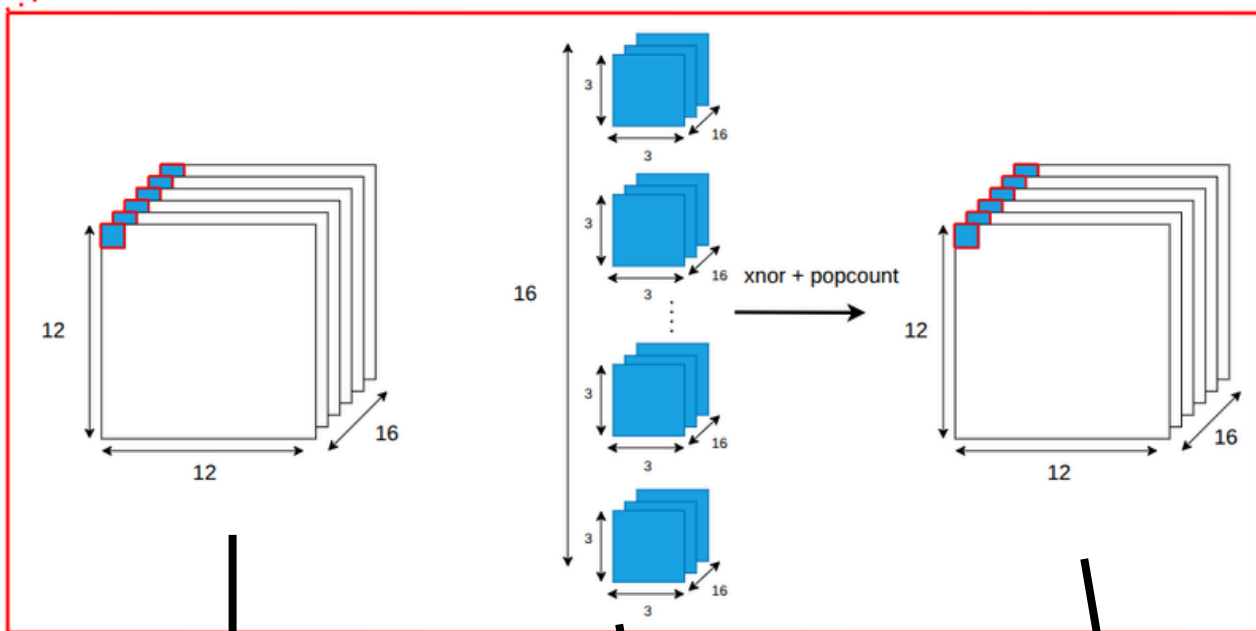
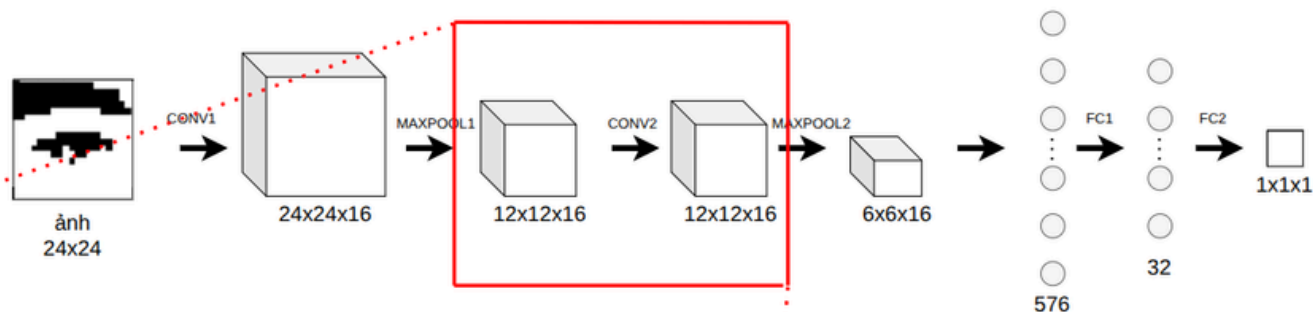






***CONV2***



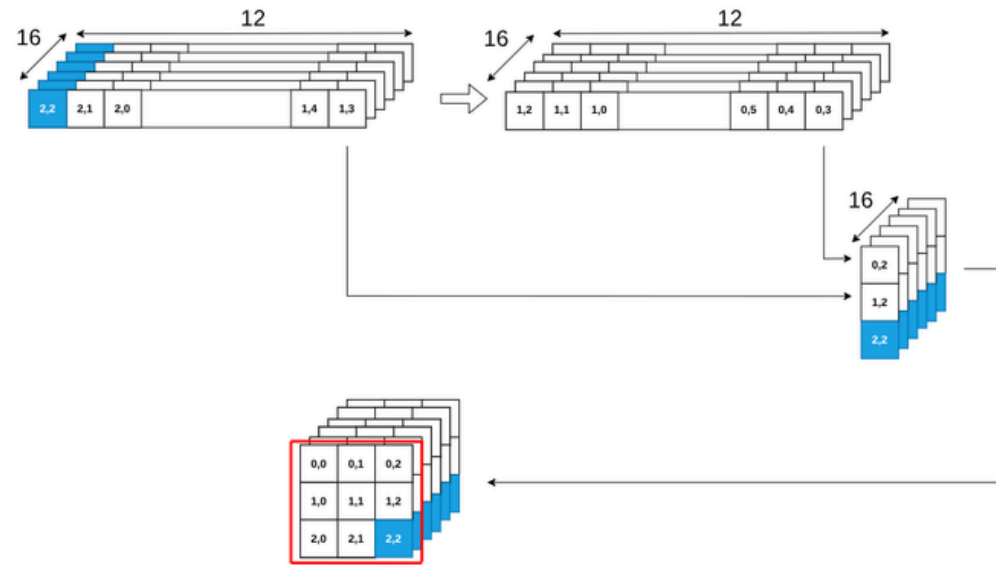
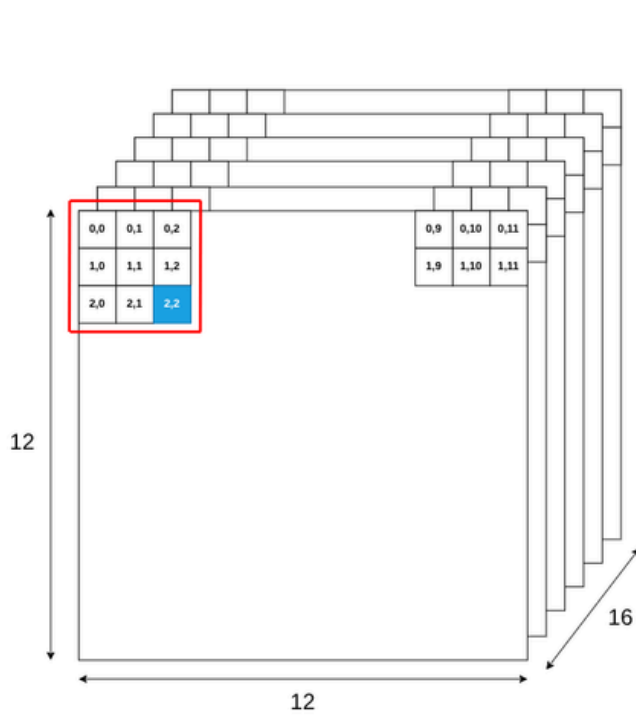


load ifm từ bram  
addr 648 -> 791

load wgt từ ip gowin  
địa chỉ từ 16 đến 271

store ofm vào  
bram địa chỉ từ  
792 đến 935

## • CONV2:

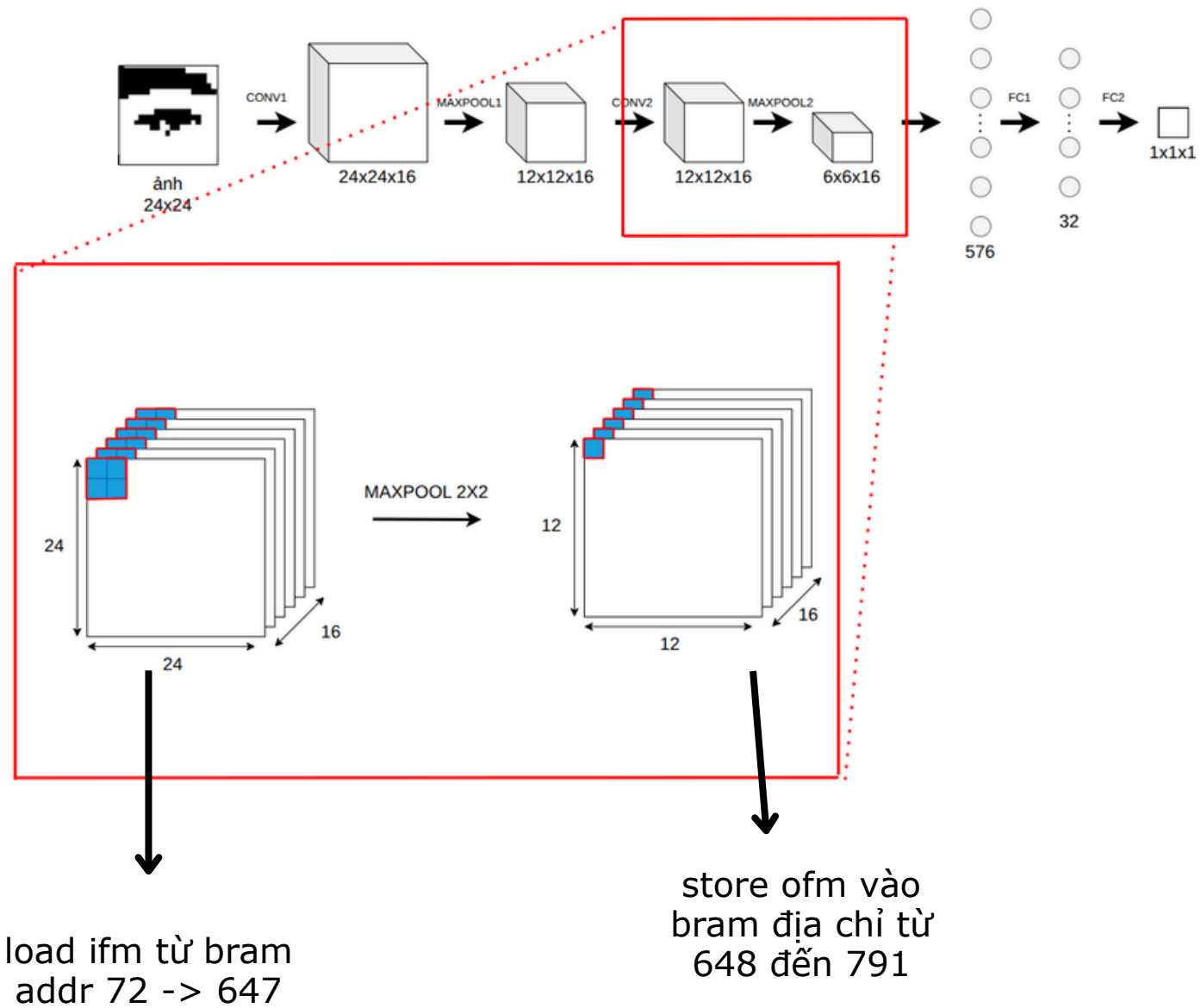


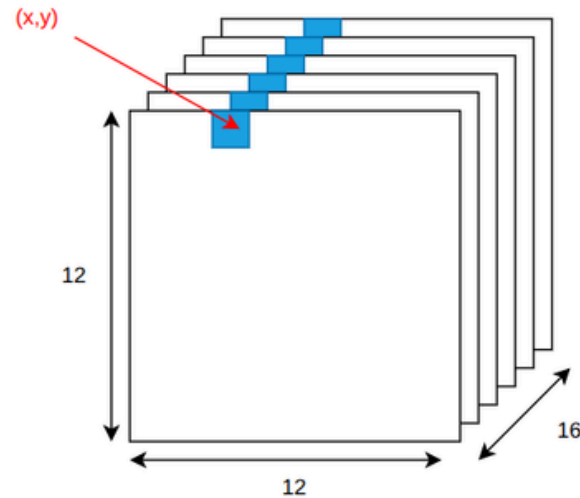
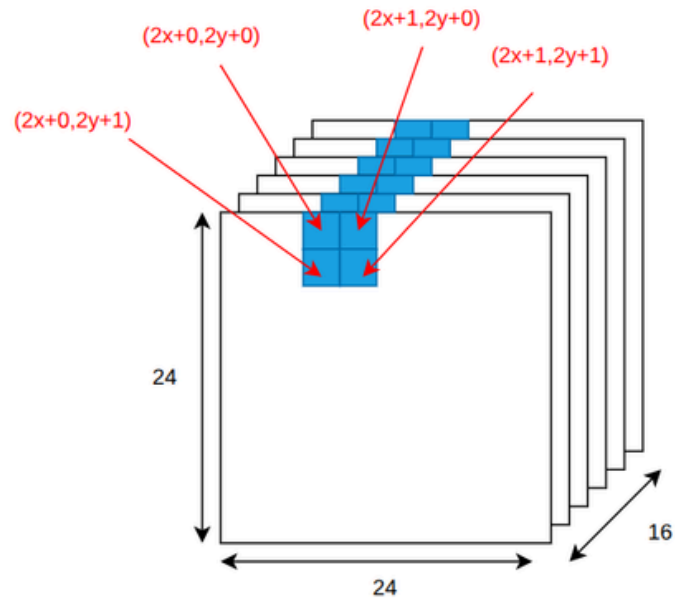


***Maxpool1***



# Maxpool1



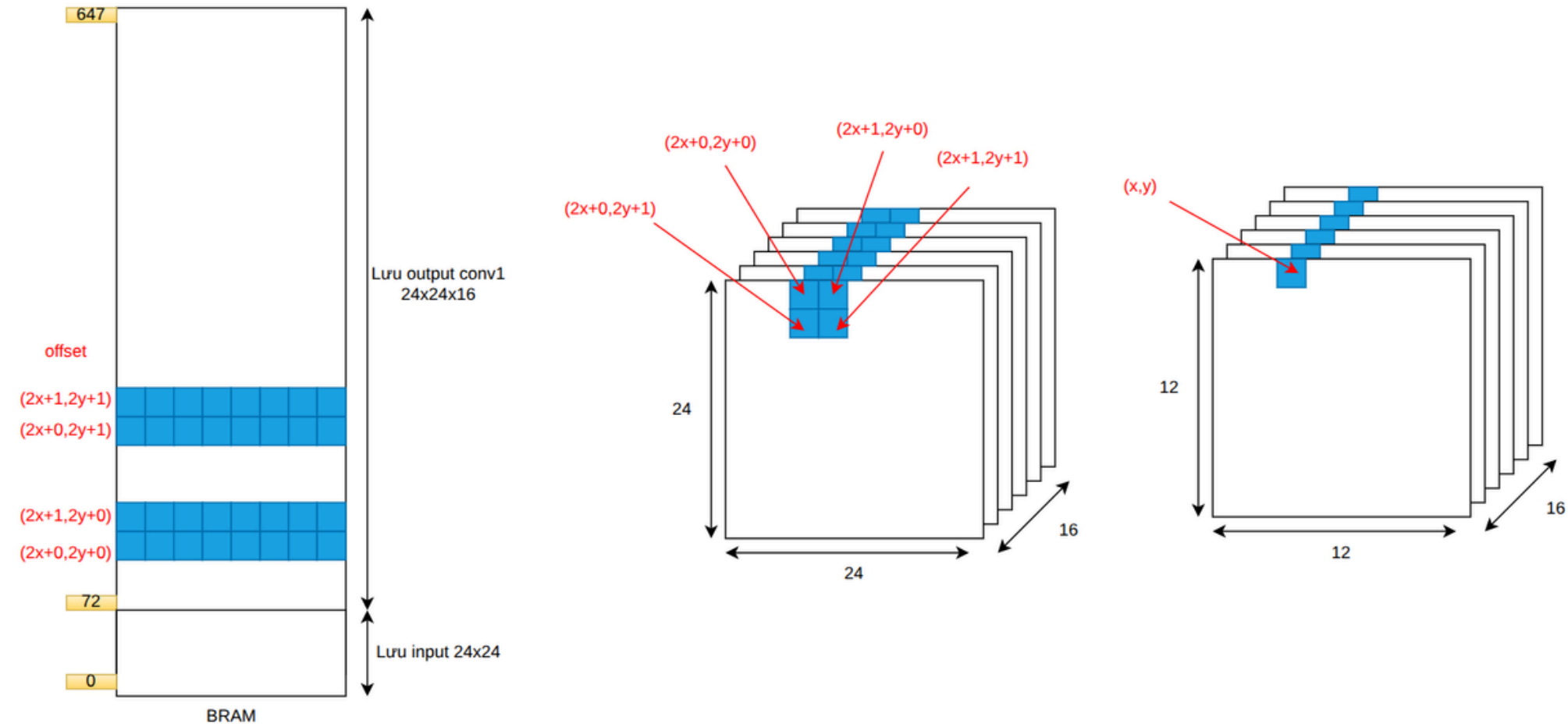


- Công thức:**

$\text{output}[x,y] = \text{MAX}(\text{pool}[2x][2y], \text{pool}[2x+1][2y], \text{pool}[2x][2y+1], \text{pool}[2x+1][2y+1])$

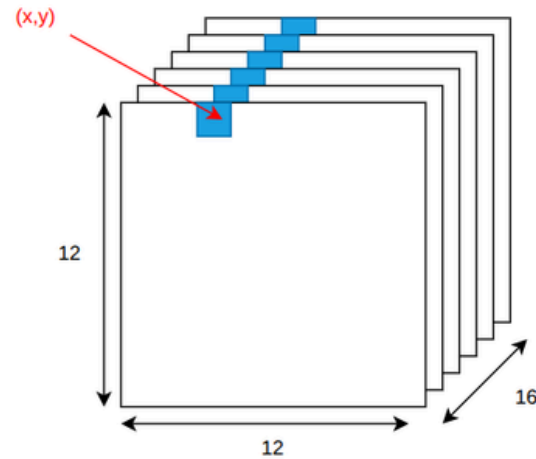
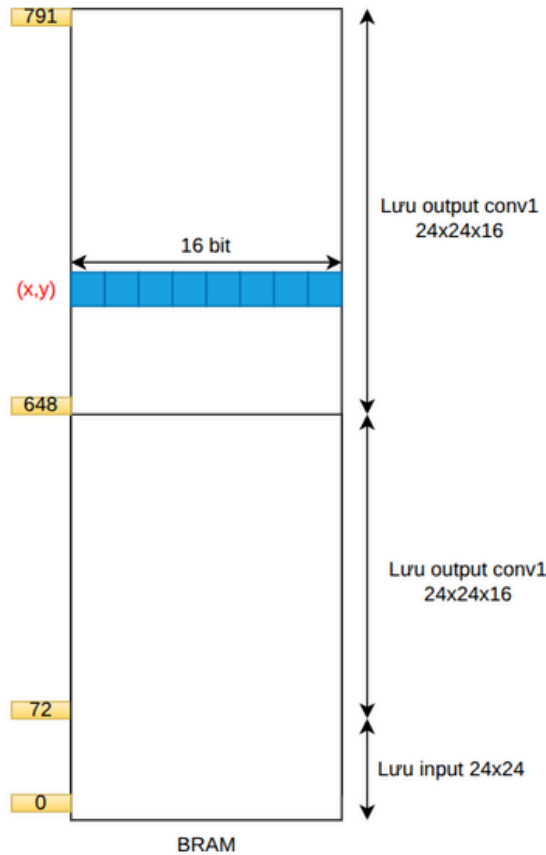
- Mà mỗi pixel là nhị phân 0 hoặc 1, nên ta có:**

$\text{output}[x,y] = \text{pool}[2x][2y] \mid \text{pool}[2x+1][2y] \mid \text{pool}[2x][2y+1] \mid \text{pool}[2x+1][2y+1]$



## • Công thức load từ BRAM để tính maxpool:

```
uint16_t buffer[3];
For(x = 0; x < 12; x++)
  For(y = 0; y < 12; y++)
    For(i = 0; i < 4; i++)
      j = (i >> 1) & 1; // 0 0 1 1
      k = i & 1; // 0 1 0 1
      addr = 72 + (2y+j)x24+(2x+k);
      if(i < 3) buffer[i] = BRAM[addr];
      else output[x][y] = buffer[0] | buffer[1] | buffer[2] | BRAM[addr]
```



- Công thức store lại vào bram sau maxpool:**

```
uint16_t buffer[3];
For(x = 0, x < 12; x++)
    For(y = 0; y < 12; y++)
        addr = 648 + yx12 + x;
        BRAM[addr] = output[x][y];
```

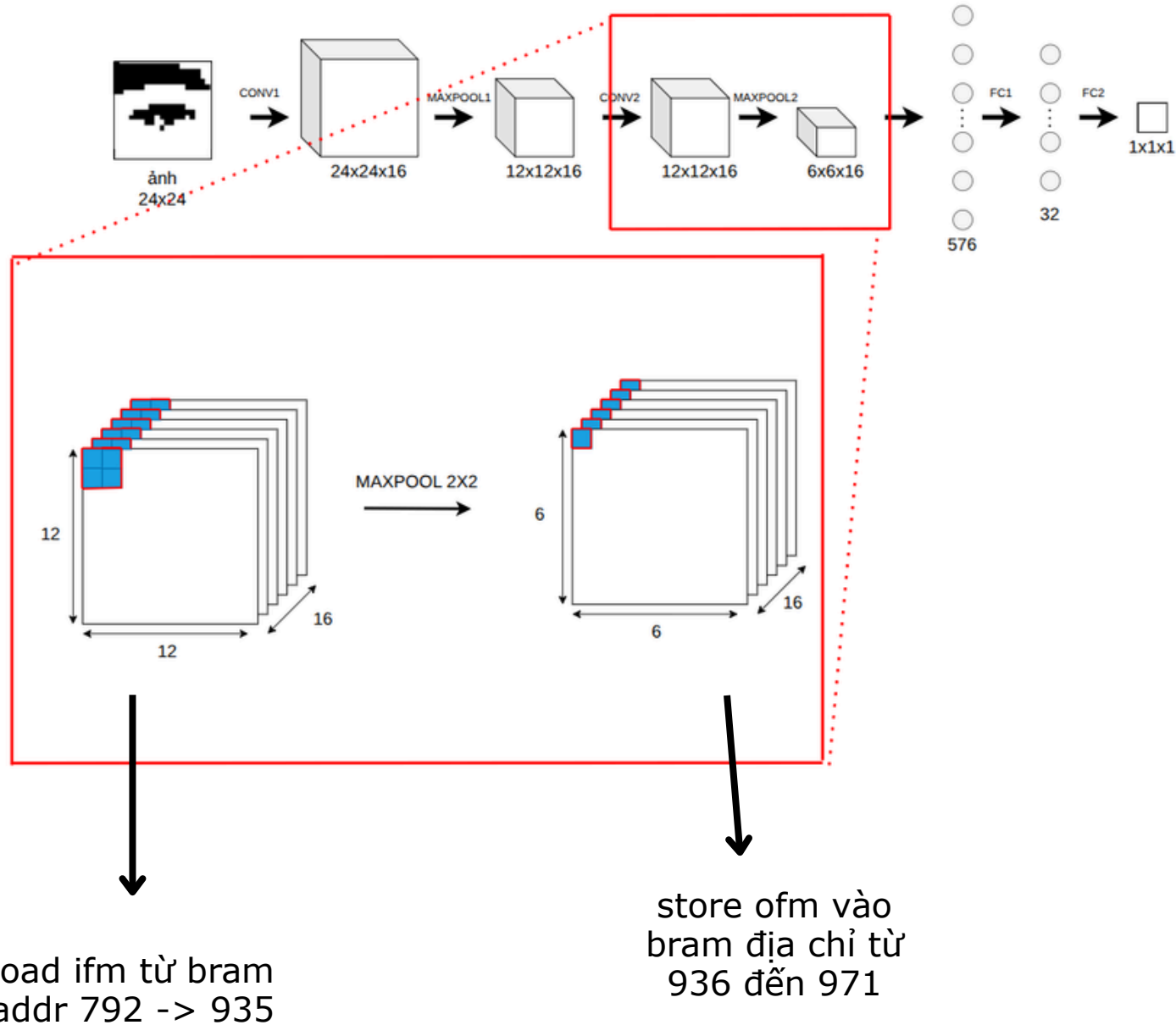


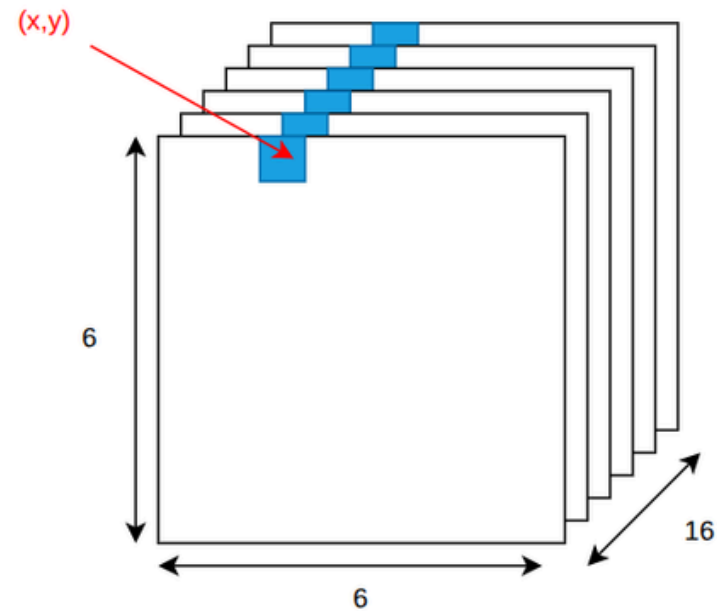
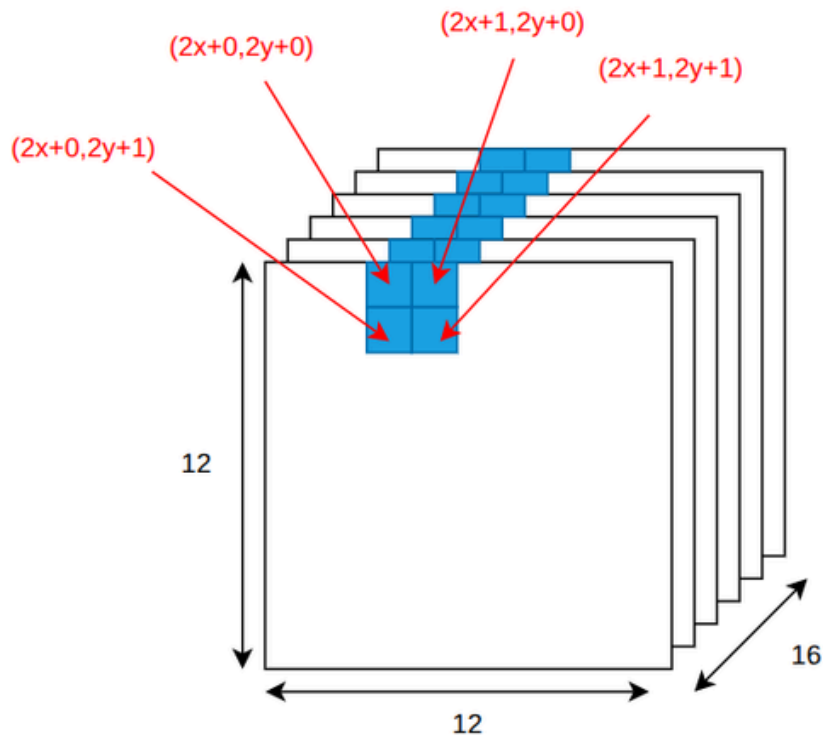


*Maxpool2*



# Maxpool2





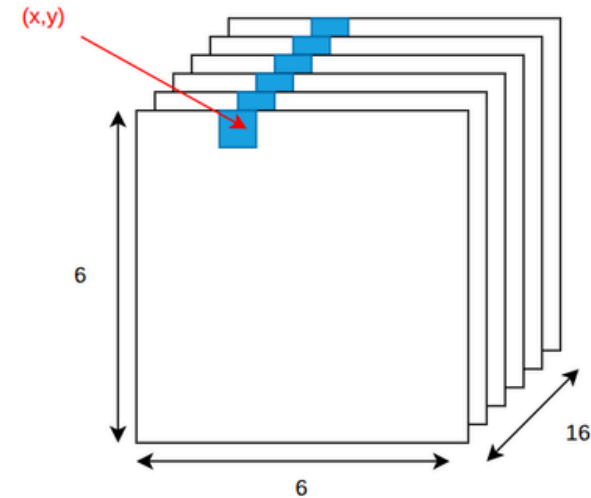
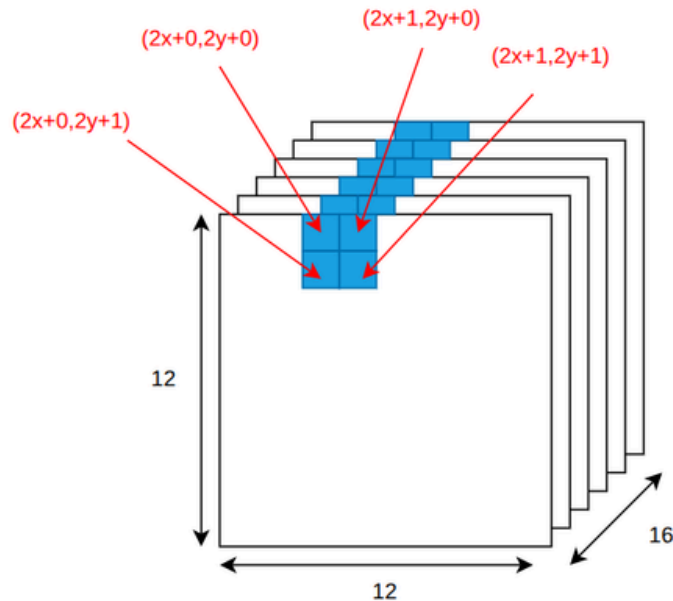
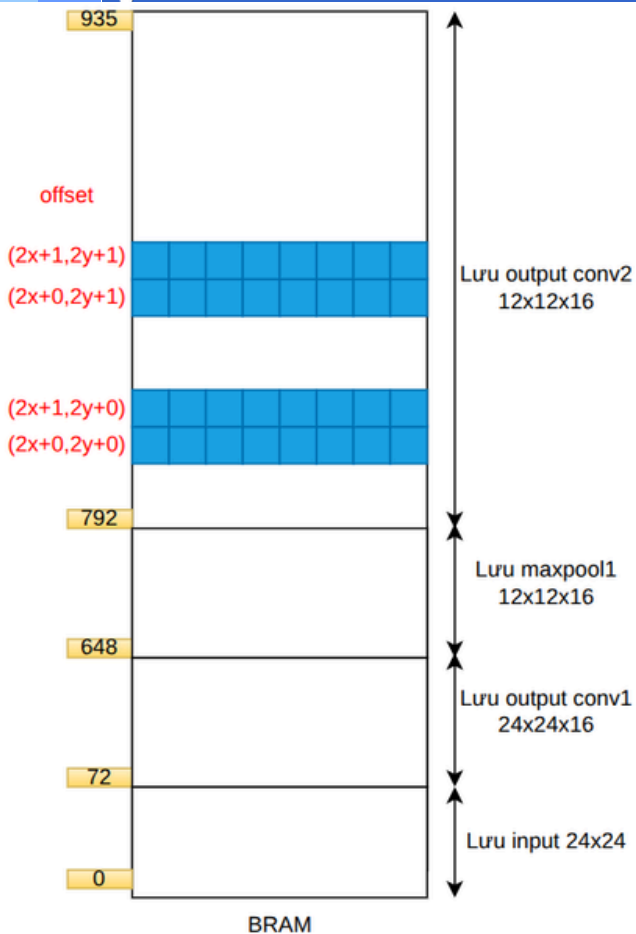
- Công thức:**

$$\text{output}[x,y] = \text{MAX}(\text{pool}[2x][2y], \text{pool}[2x+1][2y], \text{pool}[2x][2y+1], \text{pool}[2x+1][2y+1])$$

- Mà mỗi pixel là nhị phân 0 hoặc 1, nên ta có:**

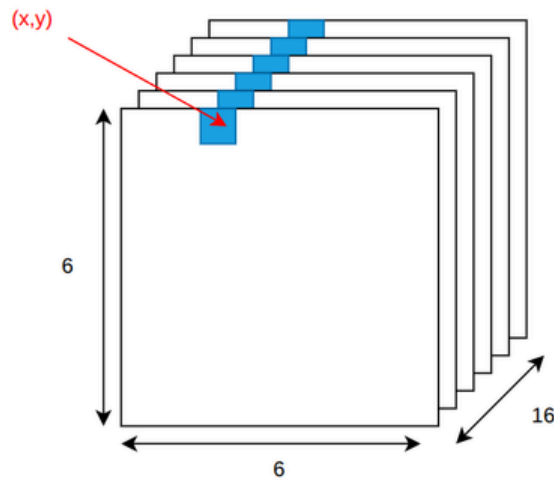
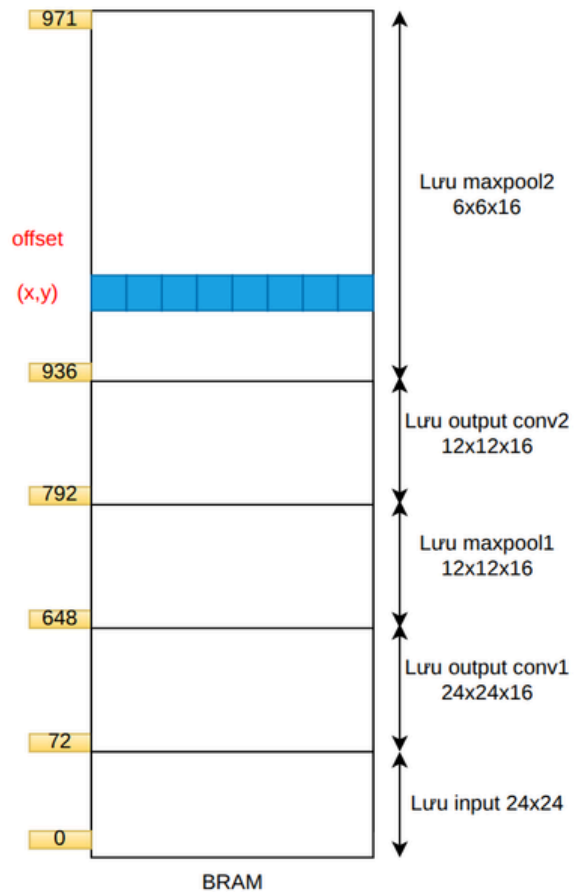
$$\text{output}[x,y] = \text{pool}[2x][2y] \mid \text{pool}[2x+1][2y] \mid \text{pool}[2x][2y+1] \mid \text{pool}[2x+1][2y+1]$$

# Maxpool2



```
uint16_t buffer[3];
For( $x = 0, x < 6; x++$ )
  For( $y = 0; y < 6; y++$ )
    For( $i = 0; i < 4; i++$ )
       $j = (i >> 1) \& 1; // 0 0 1 1$ 
       $k = i \& 1; // 0 1 0 1$ 
       $addr = 792 + (2y+j) \times 12 + (2x+k);$ 
      if( $i < 3$ )  $buffer[i] = BRAM[addr];$ 
      else  $output[x][y] = buffer[0] | buffer[1] | buffer[2] | BRAM[addr]$ 
```

# Maxpool2



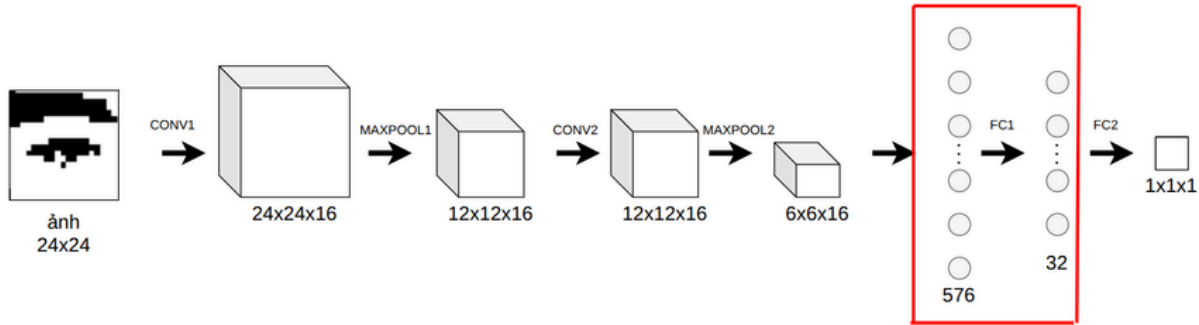
```
uint16_t buffer[3];
For(x = 0, x < 6; x ++ )
  For (y = 0; y < 6 ; y ++ )
    addr = 936 + yx6 + x;
    BRAM[addr] = output[x][y];
```



***FC1 & FC2***

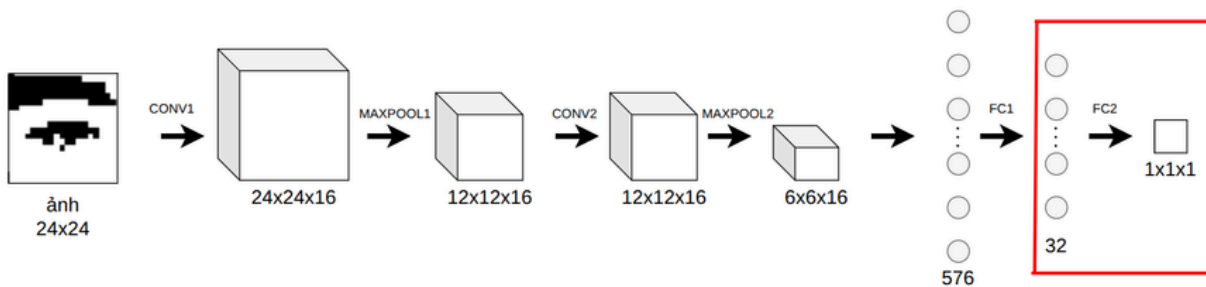


## • Fully connected 1:



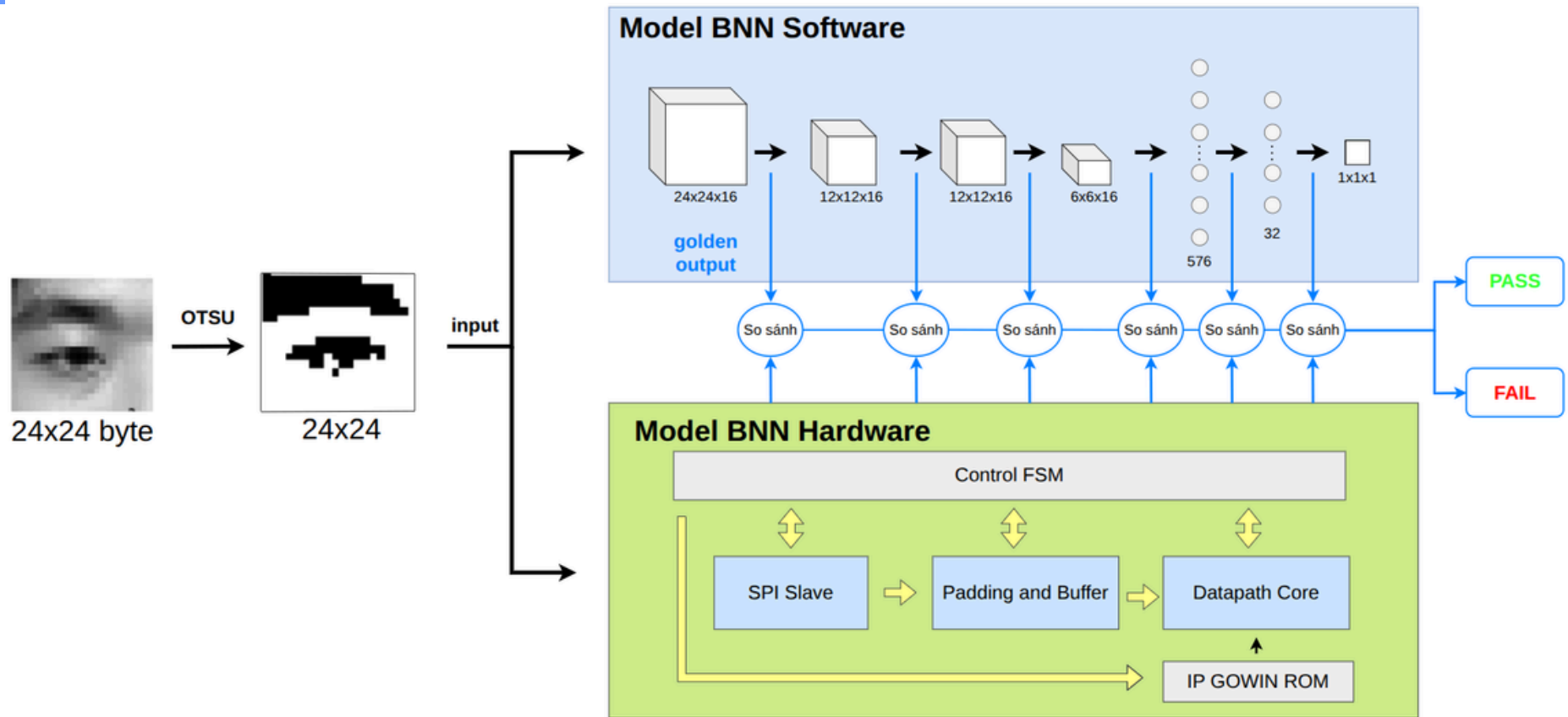
- + Chia 576 thành 36 tile, mỗi tile tính 16 bit.
- + Load ifm : địa chỉ 936 -> 971.
- + store ofm : địa chỉ 972 -> 973.

## • Fully connected 2:



- + Chia 32 thành 2 tile, mỗi tile tính 16 bit.
- + Load ifm : địa chỉ 972 -> 973.

# 5. Kiểm thử hệ thống



- **Phương pháp Kiểm thử :** Áp dụng cơ chế đối chiếu kết quả từng lớp giữa Model trên C và RTL.
- **Tập dữ liệu Kiểm thử:**
- **Tiêu chuẩn Đánh giá:**
  - + **PASS:** Khi đầu ra của RTL khớp hoàn toàn 100% với golden output từng layer.
  - + **FAIL:** Khi ít nhất 1 kết quả sai trong hệ thống testbench.



## 6. KẾT QUẢ VÀ HƯỚNG PHÁT TRIỂN

|                      |                         |                                                                                                                           |
|----------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| I. KẾT QUẢ ĐẠT ĐƯỢC  | Lỗi AI Phần cứng (BNN)  | Triển khai thành công BNN trên FPGA Gowin. Sử dụng XNOR/Popcount thay cho bộ nhân cứng. Tài nguyên cực thấp (<1584 LUTs). |
|                      | Kiểm thử (Verification) | Testbench <b>PASS 100%</b> (Bit-true matching) so với Golden C-Model.                                                     |
|                      | Tích hợp Hệ thống       | Pipeline (PC ↔ STM32 ↔ FPGA ↔ SONiX) hoạt động thời gian thực ổn định.                                                    |
| II. HƯỚNG PHÁT TRIỂN | Hệ thống SoC Độc lập    | Nối trực tiếp Camera vào FPGA (bỏ PC). Chuyển đổi khối cắt ROI và tính Otsu thành IP Core chạy phần cứng.                 |
|                      | Độ bao phủ thuật toán   | Huấn luyện mạng AI với dữ liệu khó (ban đêm, đeo kính). Tích hợp thêm module dò mốc khuôn mặt (Landmark Tracking).        |
|                      | Sản phẩm hóa & IoT      | Tích hợp FPGA và MCU lên 1 board PCB (Dashcam). Thêm module 4G/Wi-Fi để cảnh báo đồng bộ lên App hoặc Server.             |